LabVIEW
First Lecture

Features

> Uses Graphic Symbols

> Created by National Instruments

> Virtual Instruments (VIs)

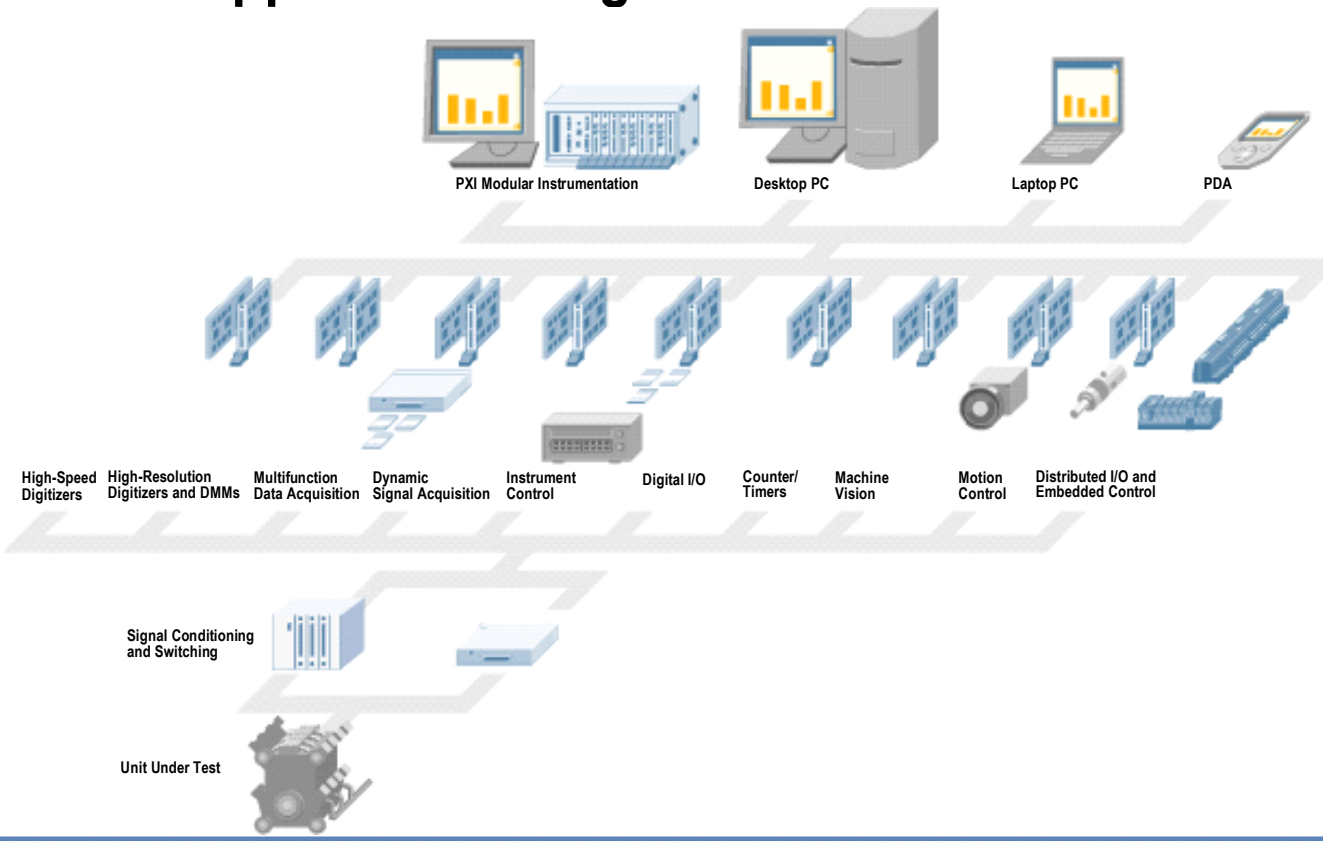
> Extensive Library of VIs

Virtual Instrumentation Applications

- **Design**
  - Signal and Image Processing
  - Embedded System Programming
    - (PC, DSP, FPGA, Microcontroller)
  - Simulation and Prototyping
  - And more…
- **Control**
  - Automatic Controls and Dynamic Systems
  - Mechatronics and Robotics
  - And more…
- **Measurements**
  - Circuits and Electronics
  - Measurements and Instrumentation
  - And more…

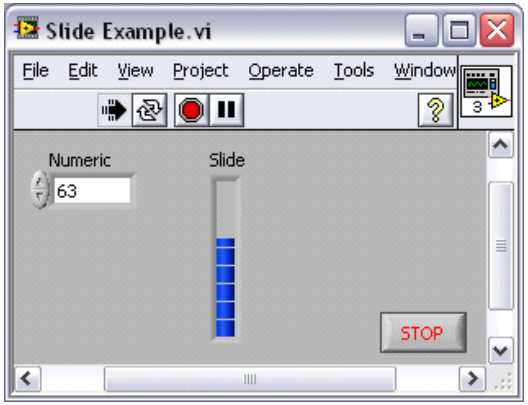
A single graphical development platform

The NI Approach – Integrated Hardware Platforms



LabVIEW Programs Are Called Virtual Instruments (VIs)

Each VI has 2 Windows

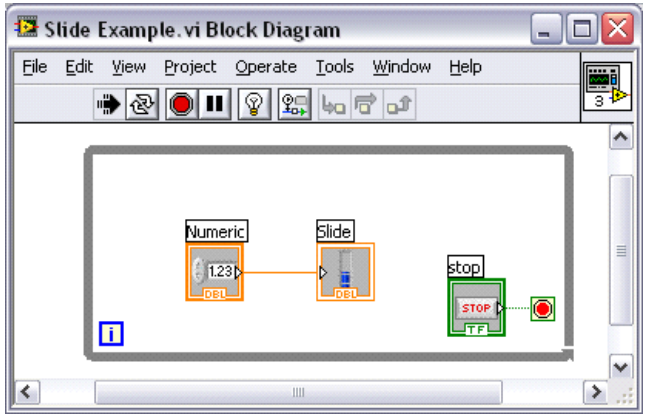


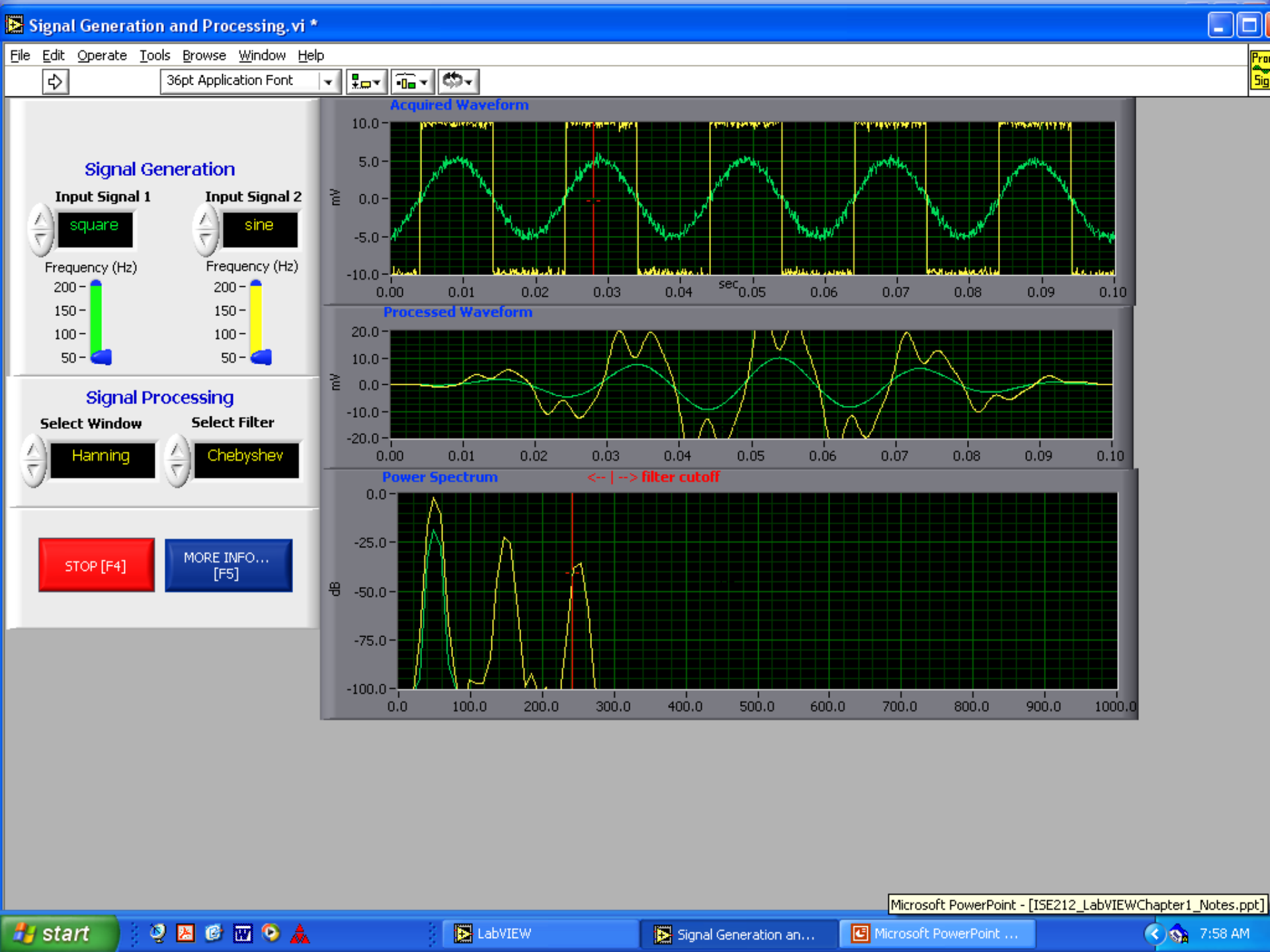
Front Panel

- User Interface (UI)
  - Controls = Inputs
  - Indicators = Outputs

Block Diagram

- Graphical Code
  - Data travels on wires from controls through functions to indicators
  - Blocks execute by Dataflow




Signal Generation and Processing.vi *
File Edit Operate Tools Browse Window Help
36pt Application Font
Signal Generation
Input Signal 1
square
Frequency (Hz)
200
150
100
50
Input Signal 2
sine
Frequency (Hz)
200
150
100
50
Signal Processing
Select Window
Hanning
Select Filter
Chebyshev
STOP [F4]
MORE INFO... [F5]
Acquired Waveform
mV
10.0
5.0
0.0
-5.0
-10.0
0.00 0.01 0.02 0.03 0.04 sec 0.05 0.06 0.07 0.08 0.09 0.10
Processed Waveform
mV
20.0
10.0
0.0
-10.0
-20.0
0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10
Power Spectrum
<-- | --> filter cutoff
dB
0.0
-25.0
-50.0
-75.0
-100.0
0.0 100.0 200.0 300.0 400.0 500.0 600.0 700.0 800.0 900.0 1000.0
Microsoft PowerPoint - [ISE212_LabVIEWChapter1_Notes.ppt]
start
LabVIEW
Signal Generation an...
Microsoft PowerPoint ...
7:58 AM

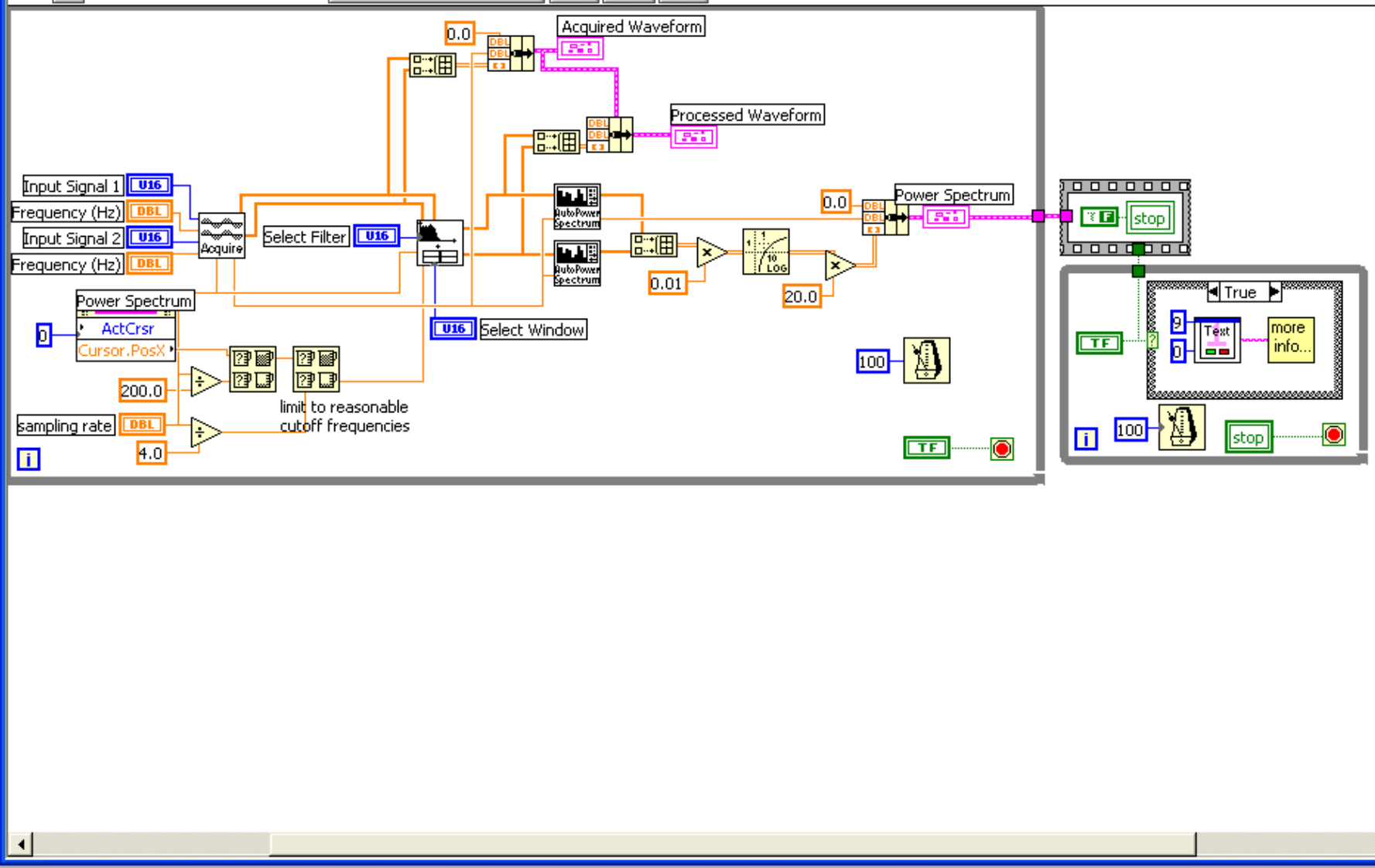
Signal Generation and Processing.vi *

Signal Generation and Processing.vi Diagram *

File Edit Operate Tools Browse Window Help

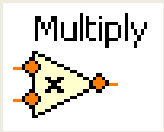
13pt Application Font

Process Signal

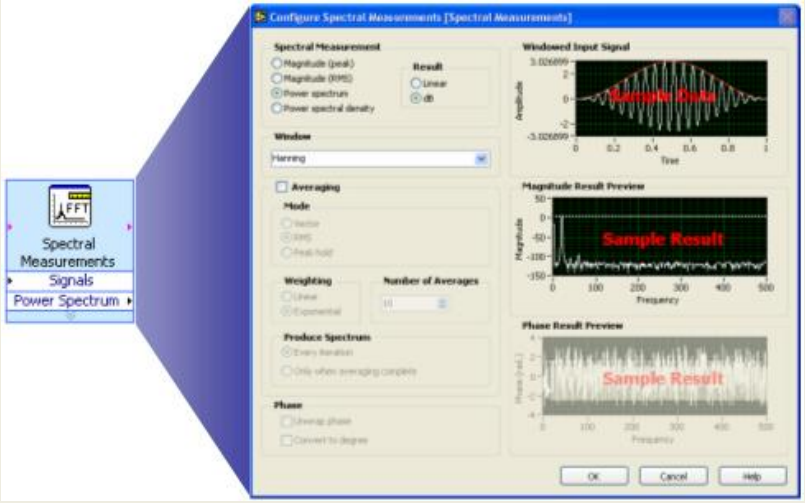


Express VIs, VIs and Functions

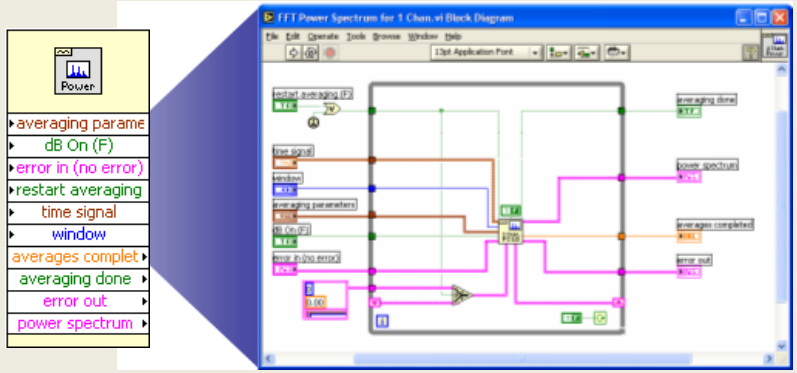
- Express VIs: interactive VIs with configurable dialog page
- Standard VIs: modularized VIs customized by wiring
- Functions: fundamental operating elements of LabVIEW; no front panel or block diagram



Function

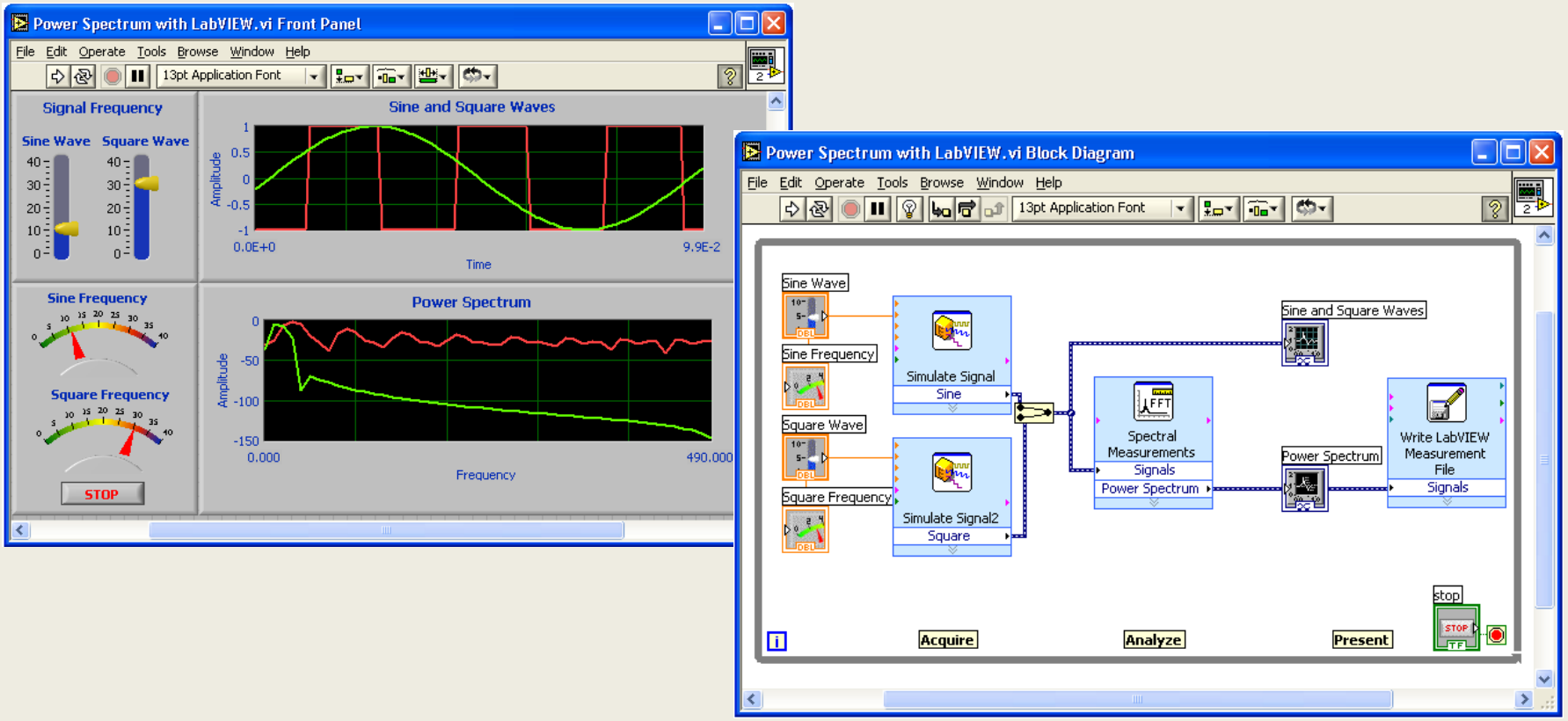


Express VI



Standard VI

Virtual Instrumentation With LabVIEW



Front Panel Toolbar

> Run / Broken Run

> Continuous Run

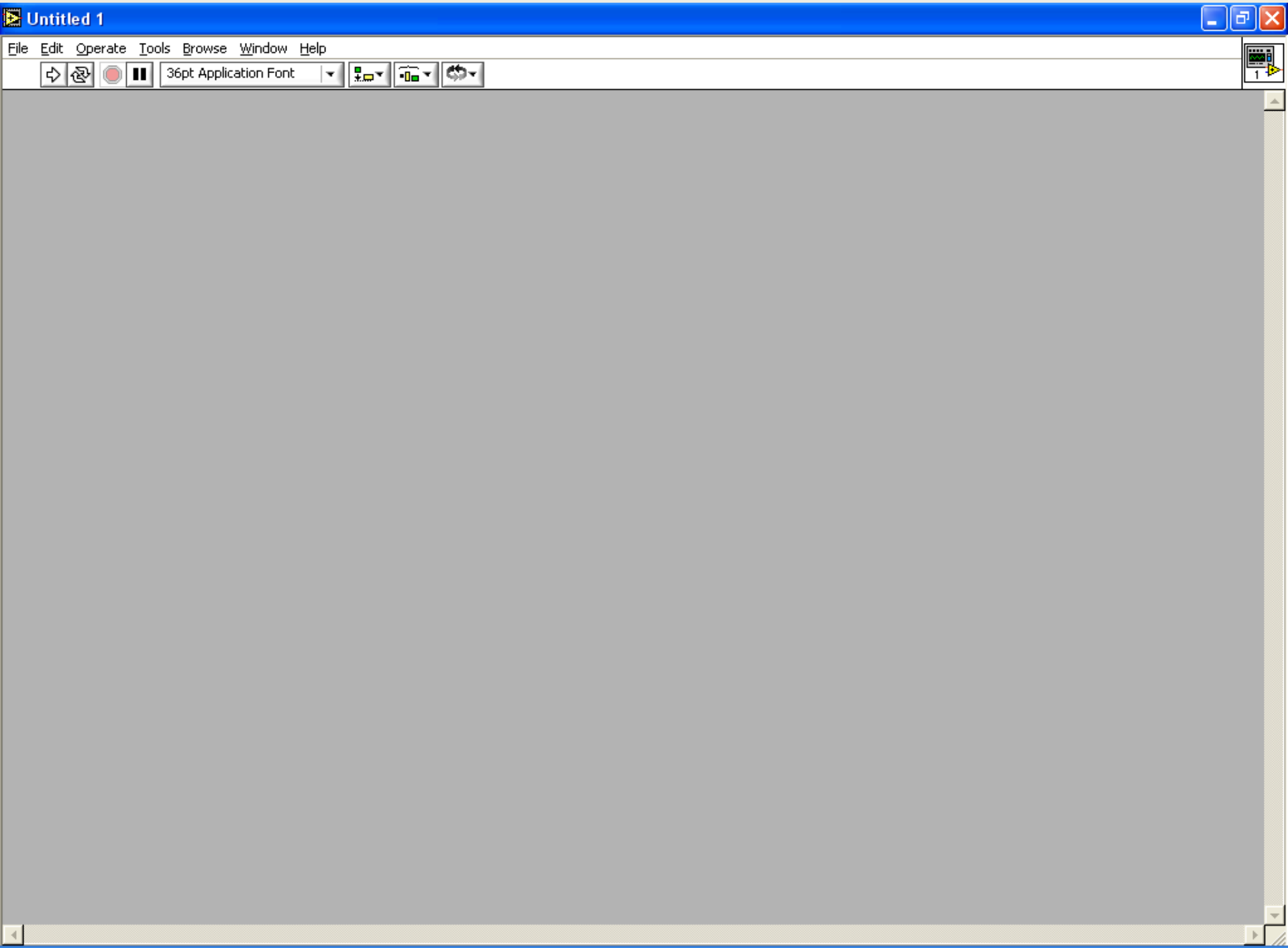
> Abort Execution

> Pause / Continue

> Font Ring

> Alignment Ring

> Distribution Ring


Untitled 1
File Edit Operate Tools Browse Window Help
36pt Application Font

Status Toolbar

Run Button

Continuous Run Button

Abort Execution

Additional Buttons on the Diagram Toolbar

Execution Highlighting Button

Retain Wire Values Button

Step Function Buttons

Block Diagram Toolbar

Additional:

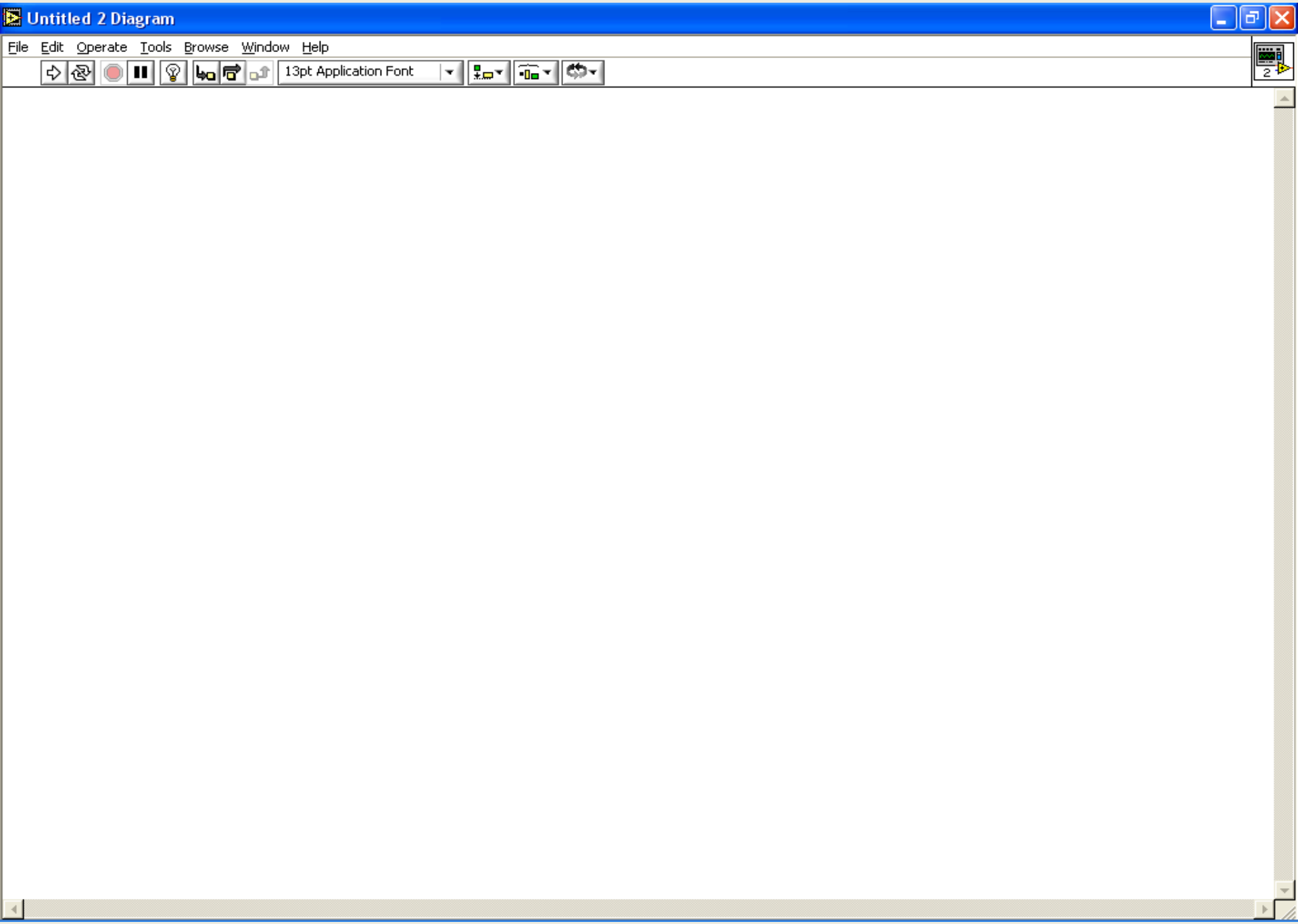
> Highlight Execution

> Step Into

> Step Over

> Step Out

> Warning Indicator


Untitled 2 Diagram
File Edit Operate Tools Browse Window Help
13pt Application Font
2

Status Toolbar

Run Button

Continuous Run Button

Abort Execution

Pause/Continue Button

Text Settings

Align Objects

Distribute Objects

Reorder

Resize front panel objects

Additional Buttons on the Diagram Toolbar

Execution Highlighting Button

Step Into Button

Step Over Button

Step Out Button

Palettes

- Tools
- Controls
- Functions

Tools Palette

- Recommended: Automatic Selection Tool
- Tools to operate and modify both front panel and block diagram objects

Automatic Selection Tool

Automatically chooses among the following tools:

Operating Tool

Positioning/Resizing Tool

Labeling Tool

Wiring Tool

Tools Palette

- **Floating Palette**
- **Used to operate and modify front panel and block diagram objects.**

Automatic Selection Tool

Operating Tool

Positioning/Resizing Tool

Labeling Tool

Wiring Tool

Shortcut Menu Tool

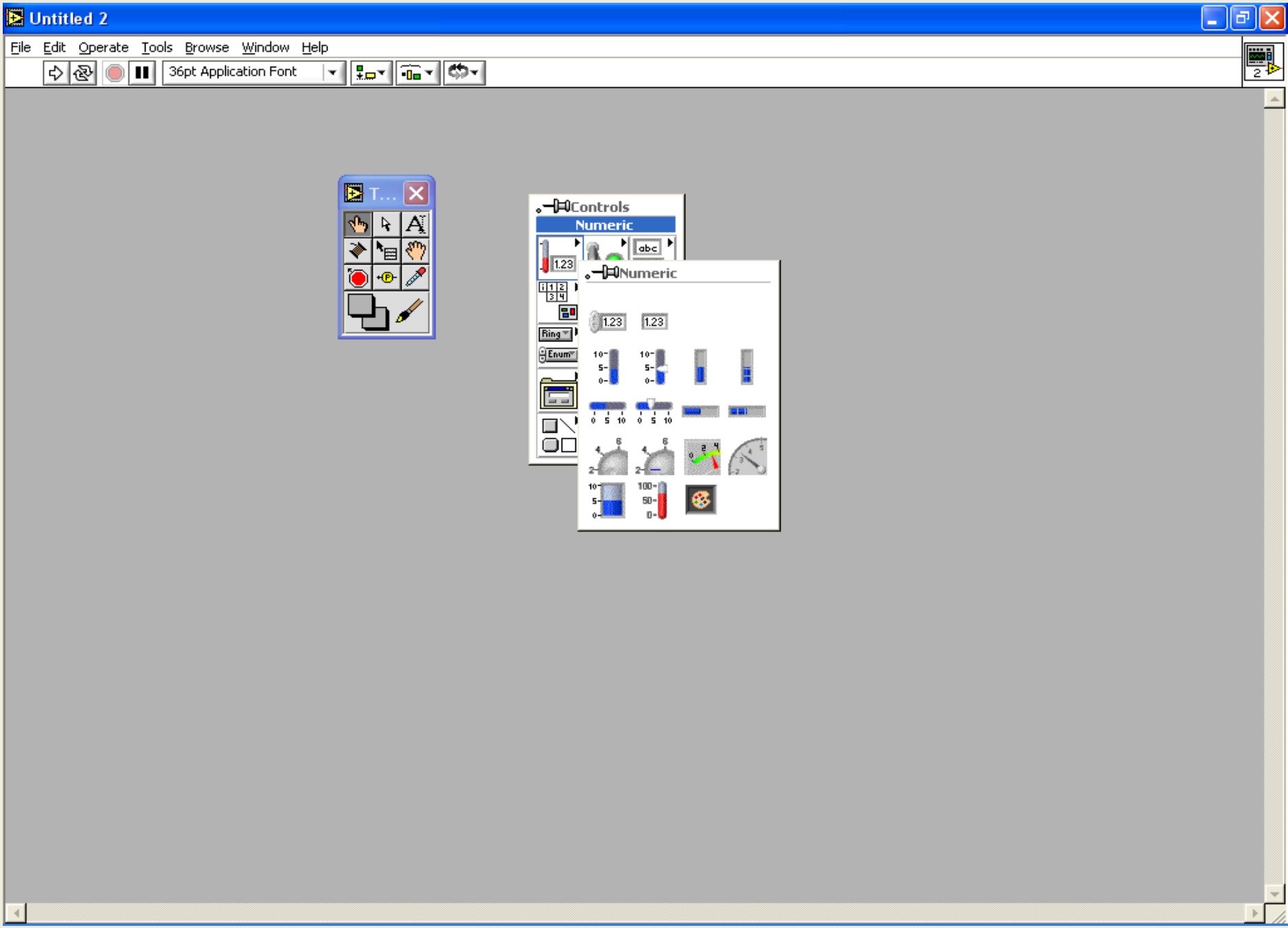
Scrolling Tool

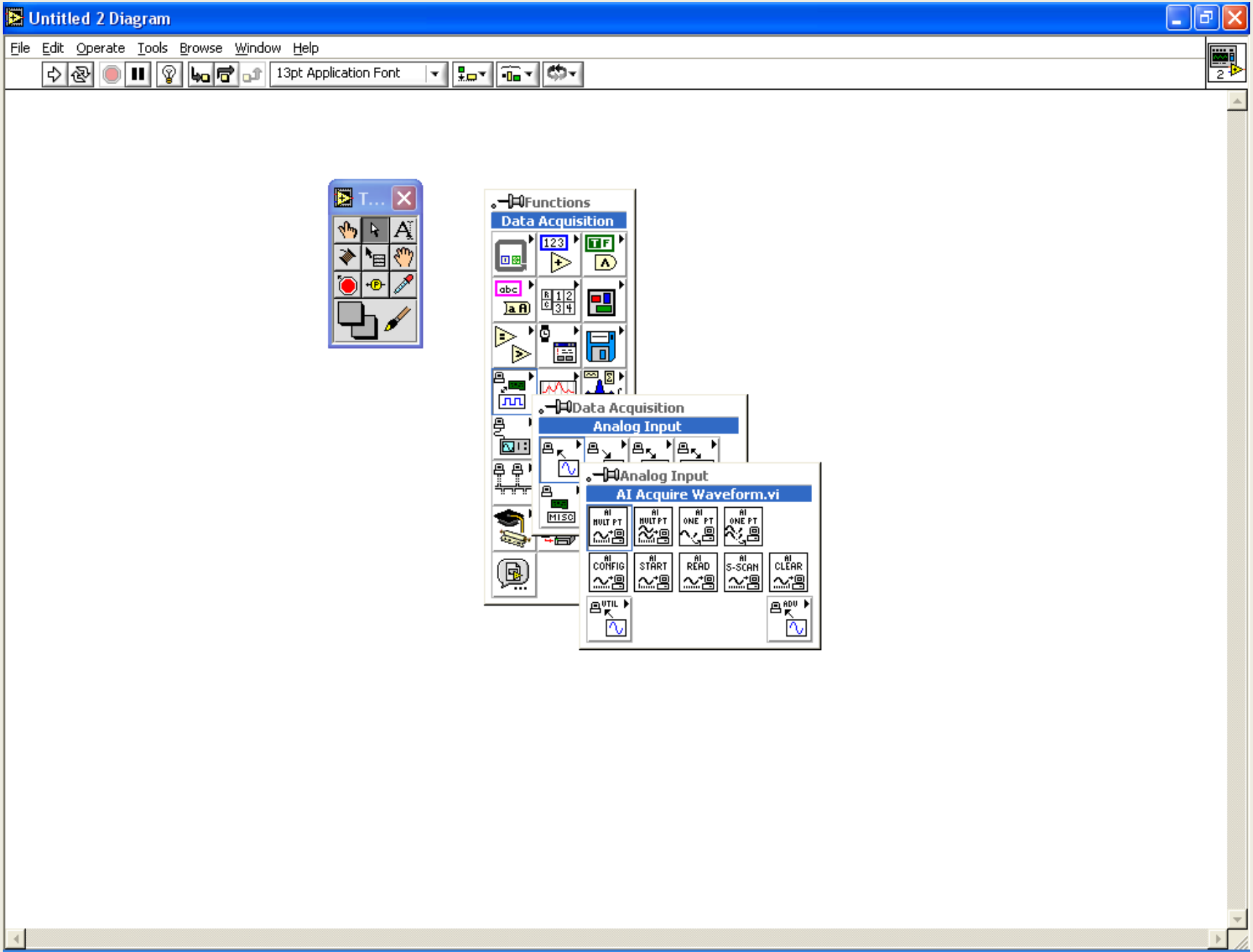
Breakpoint Tool

Probe Tool

Color Copy Tool

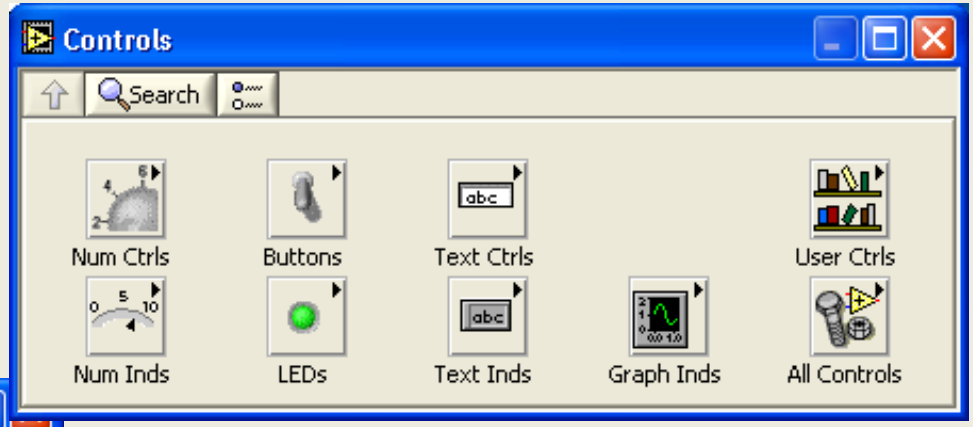
Coloring Tool

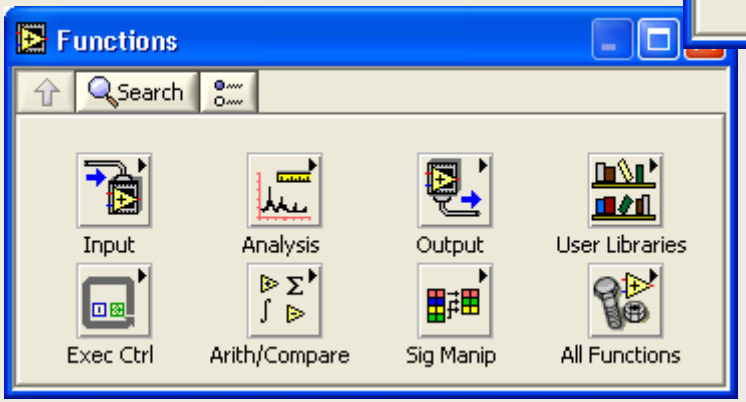

Untitled 2
File Edit Operate Tools Browse Window Help
36pt Application Font
T...
Controls
Numeric
1.23
abc
Numeric
1.23
1.23
Ring
Enum


Untitled 2 Diagram
File Edit Operate Tools Browse Window Help
13pt Application Font
Functions
Data Acquisition
Data Acquisition
Analog Input
Analog Input
AI Acquire Waveform.vi
AI MULT PT
AI MULT PT
AI ONE PT
AI ONE PT
AI CONFIG
AI START
AI READ
AI S-SCAN
AI CLEAR
UTIL
ADV

Controls and Functions Palettes

Controls Palette
(Front Panel Window)





Functions Palette
(Block Diagram Window)

Virtual Instruments

Virtual Instruments

Modular Programming

SubVIs

VI Main Components

> Front Panel

- Controls
- Indicators

> Block Diagram

- Nodes
- Terminals
- Wires

> Icon/Connector

Block Diagram Features

> Control Terminals

- Thick Borders

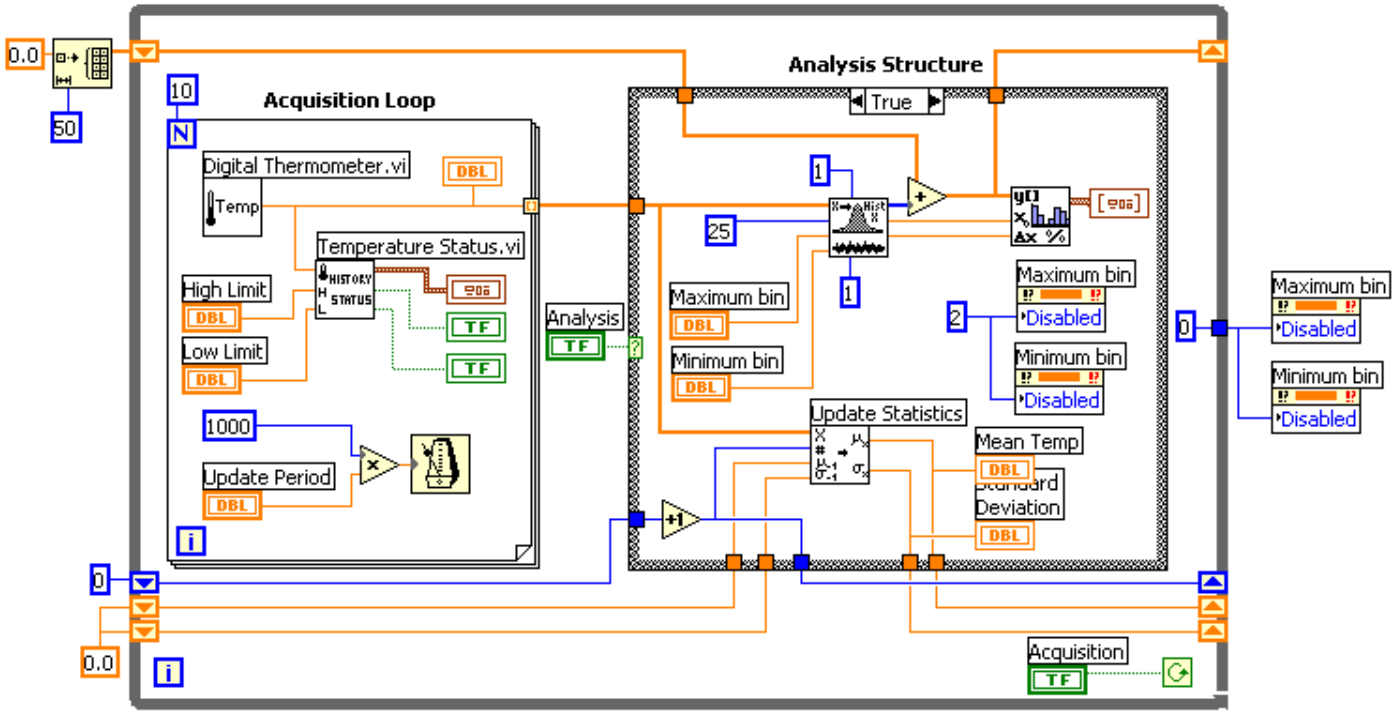
> Indicator Terminals

- Thin Borders

Temperature System Demo.vi Diagram

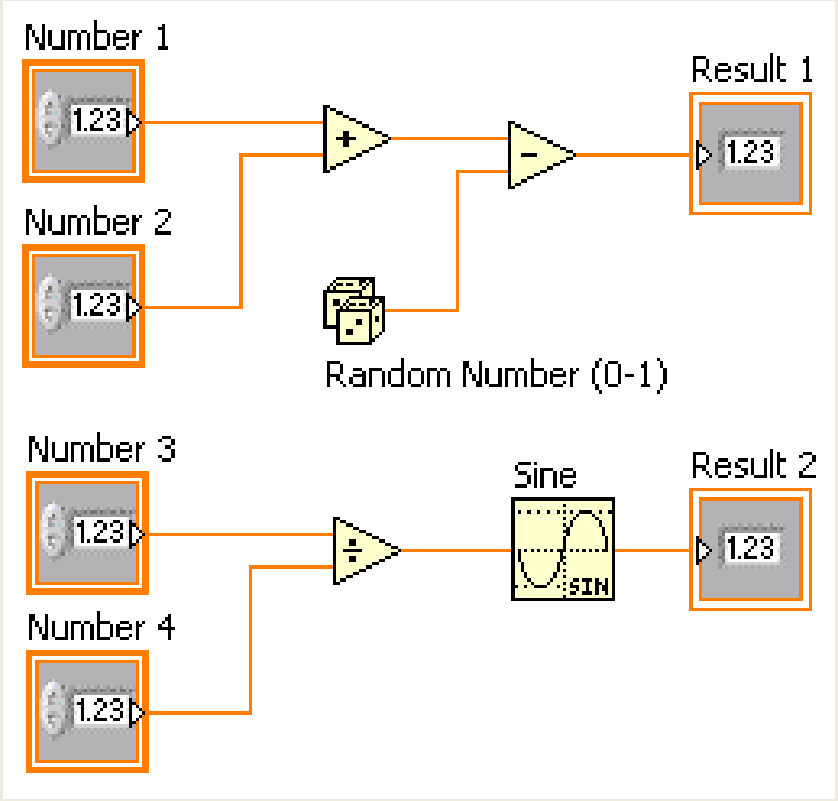
File Edit Operate Tools Browse Window Help

13pt Application Font



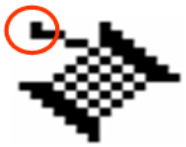
Dataflow Programming

- Block diagram executes dependent on the flow of data; block diagram does NOT execute left to right
- Node executes when data is available to ALL input terminals
- Nodes supply data to all output terminals when done

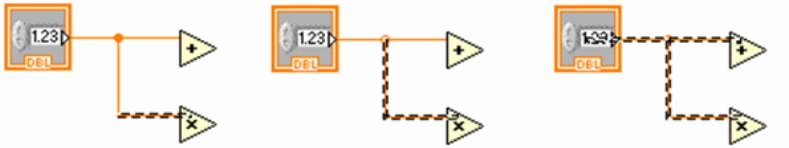


Wiring Tips – Block Diagram

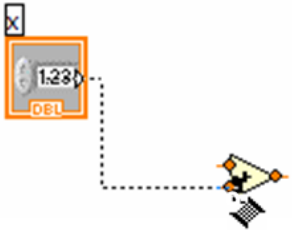
Wiring "Hot Spot"



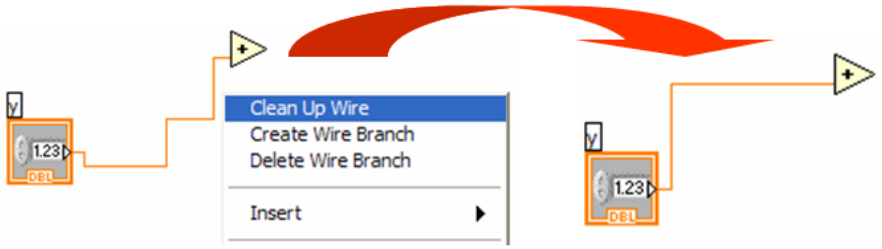
Click To Select Wires



Use Automatic Wire Routing



Clean Up Wiring



Block Diagram Features Colors:

Number – Orange

Boolean – Green

Integer – Blue

String - Pink

Wire Type:

Thin – Scalar

Thick – 1D Array

Double – 2D Array

LabVIEW
Second Lecture

Data Types in Labview

Control	Indicator	Data Type	Use	Default Values
		Single-precision, floating-point numeric	Saves memory and does not overflow the range of the numbers.	0.0
		Double-precision, floating-point numeric	Is the default format for numeric objects.	0.0
		Extended-precision, floating-point numeric	Performs differently depending on the platform. Use only when necessary.	0.0
		Complex single-precision, floating-point numeric	Same as single-precision, floating-point, with a real and an imaginary part.	0.0 + 0.0i
		Complex double-precision, floating-point numeric	Same as double-precision, floating-point, with a real and an imaginary part.	0.0 + 0.0i
		Complex extended-precision, floating-point numeric	Same as extended-precision, floating-point, with a real and an imaginary part.	0.0 + 0.0i

Data Types in Labview

		8-bit signed integer numeric	Represents whole numbers and can be positive or negative.	0
		16-bit signed integer numeric	Same as above.	0
		32-bit signed integer numeric	Same as above.	0
		64-bit signed integer numeric	Same as above.	0
		8-bit unsigned integer numeric	Represents only non-negative integers and has a larger range of positive numbers than signed integers because the number of bits is the same for both representations.	0
		16-bit unsigned integer numeric	Same as above.	0
		32-bit unsigned integer numeric	Same as above.	0
		64-bit unsigned integer numeric	Same as above.	0

Structures

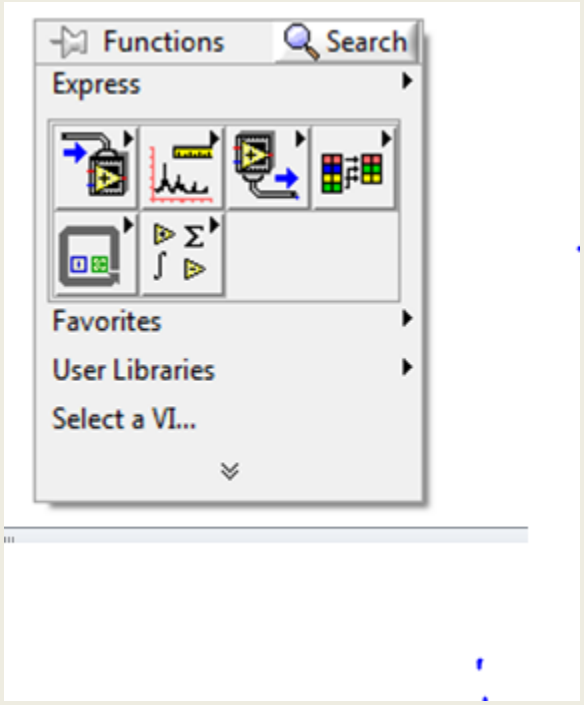
Structures

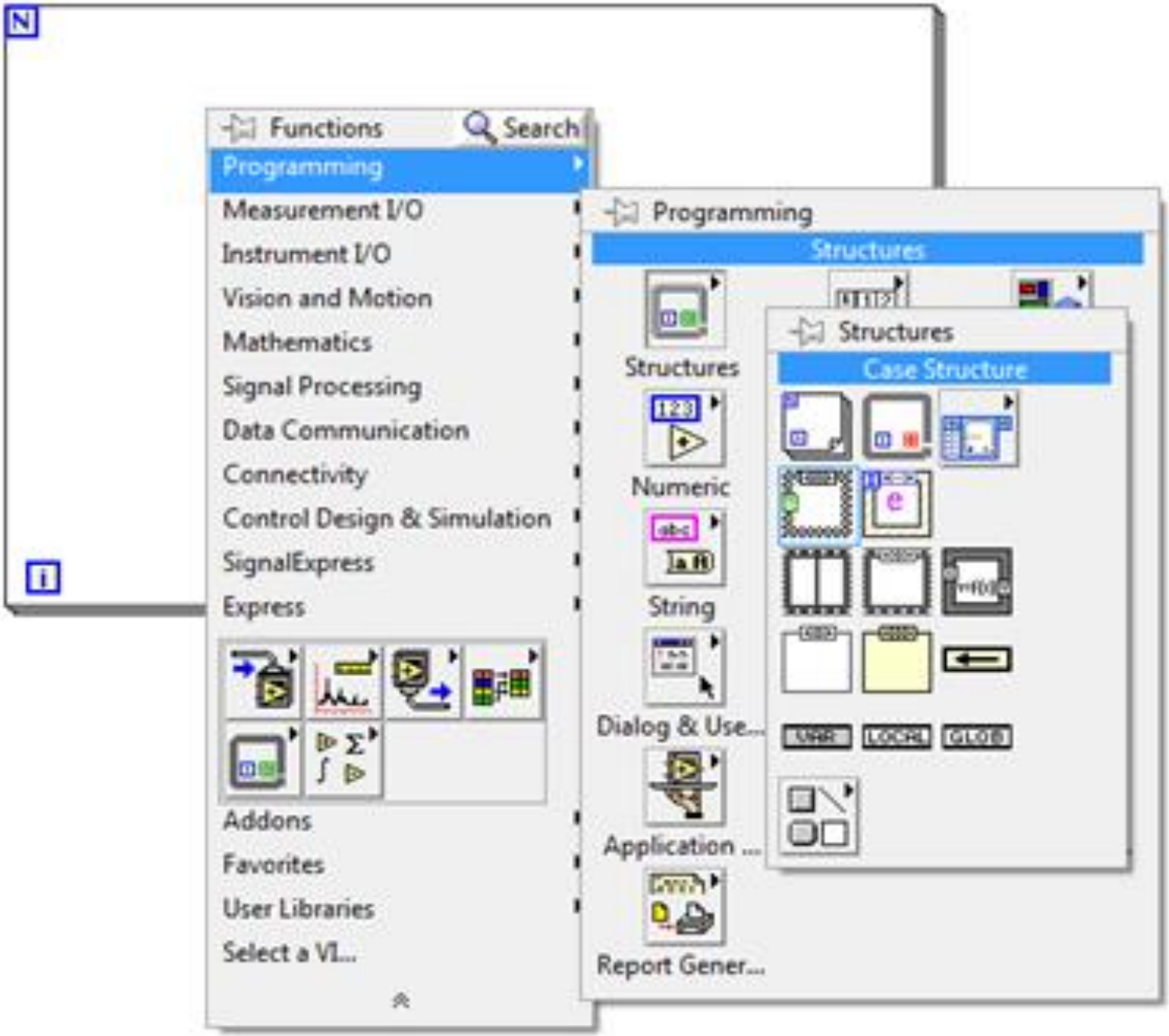
- For Loop
- While Loop
- Case
- Sequence
- Formula

For Loop

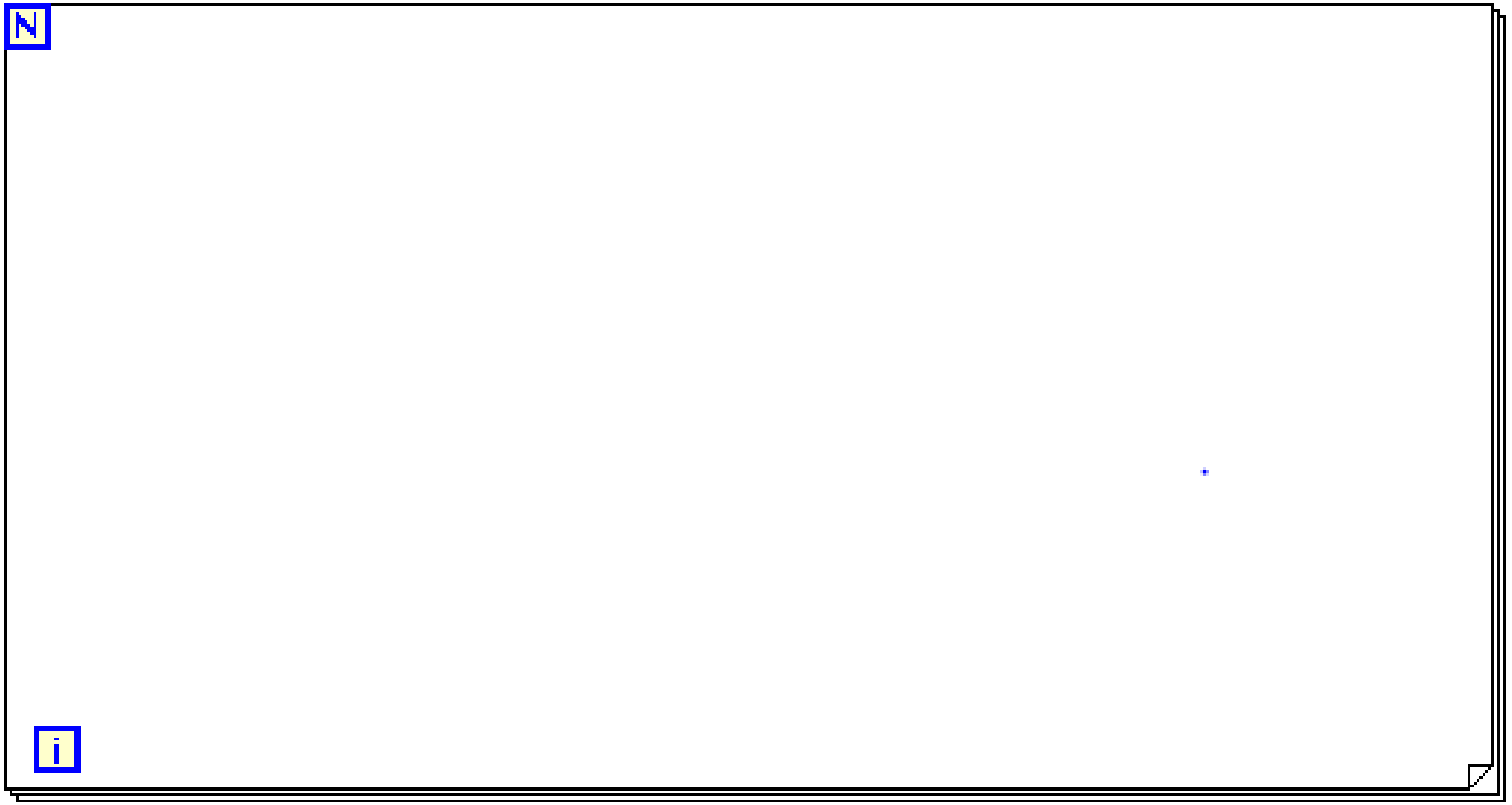
> Executes code inside borders

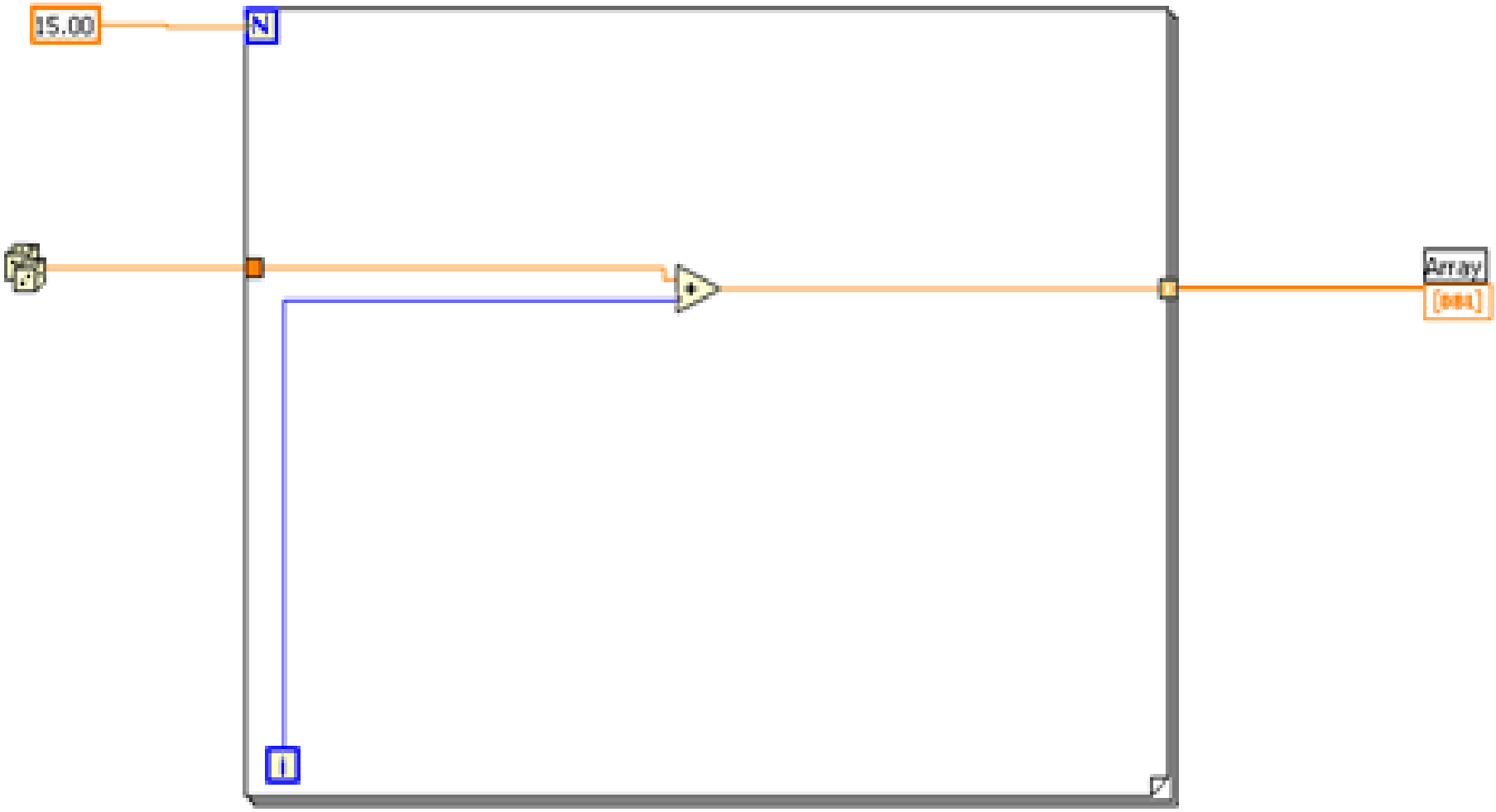
> Number of specified iterations

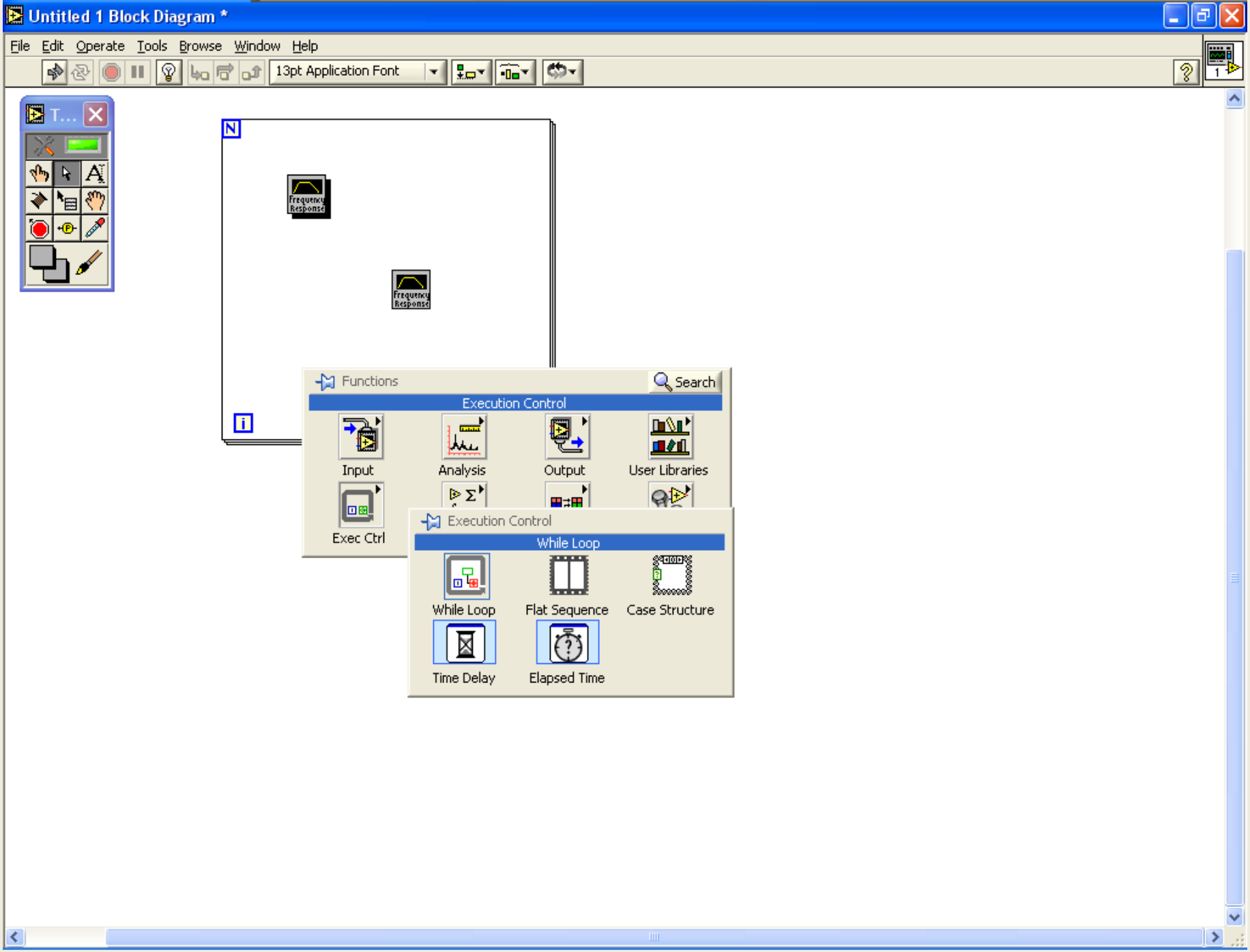

Functions
Search
Express
Favorites
User Libraries
Select a VI...


Functions
Search
Programming
Measurement I/O
Instrument I/O
Vision and Motion
Mathematics
Signal Processing
Data Communication
Connectivity
Control Design & Simulation
SignalExpress
Express
Addons
Favorites
User Libraries
Select a VI...
Programming
Structures
Structures
Numeric
String
Dialog & Use...
Application ...
Report Gener...
Structures
Case Structure
VAR
LOCAL
GLOB

For Loop




15.00
N
i
Array


Untitled 1 Block Diagram *
File Edit Operate Tools Browse Window Help
13pt Application Font
Frequency Response
Frequency Response
Functions
Search
Execution Control
Input
Analysis
Output
User Libraries
Exec Ctrl
Execution Control
While Loop
While Loop
Flat Sequence
Case Structure
Time Delay
Elapsed Time

While Loop

> Executes code inside borders until a condition is met.

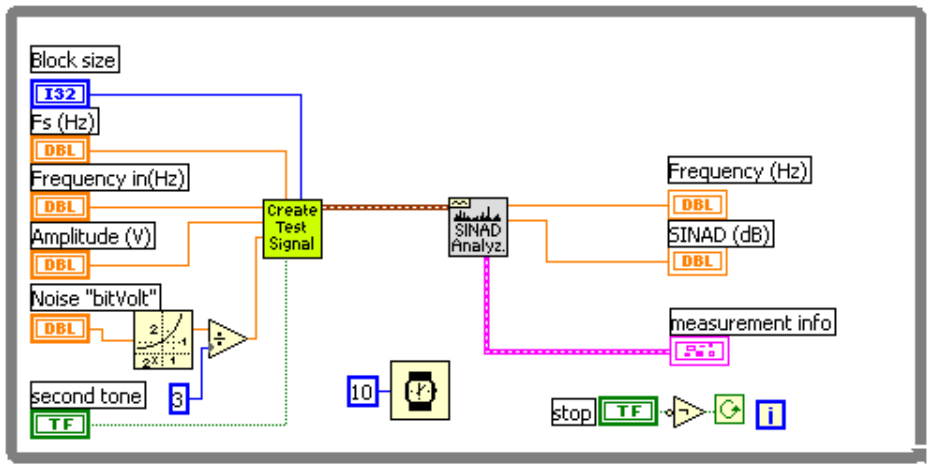
> True or False condition

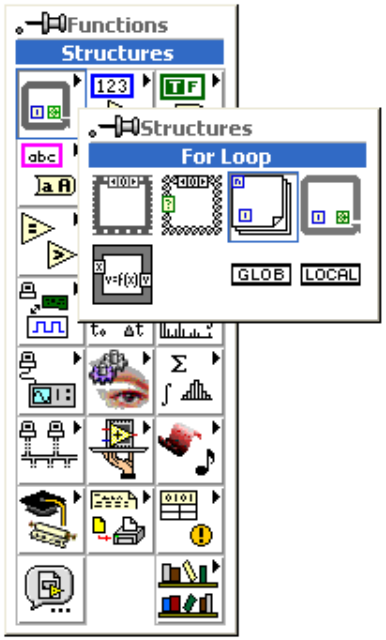
Basic SINAD Example.vi Diagram

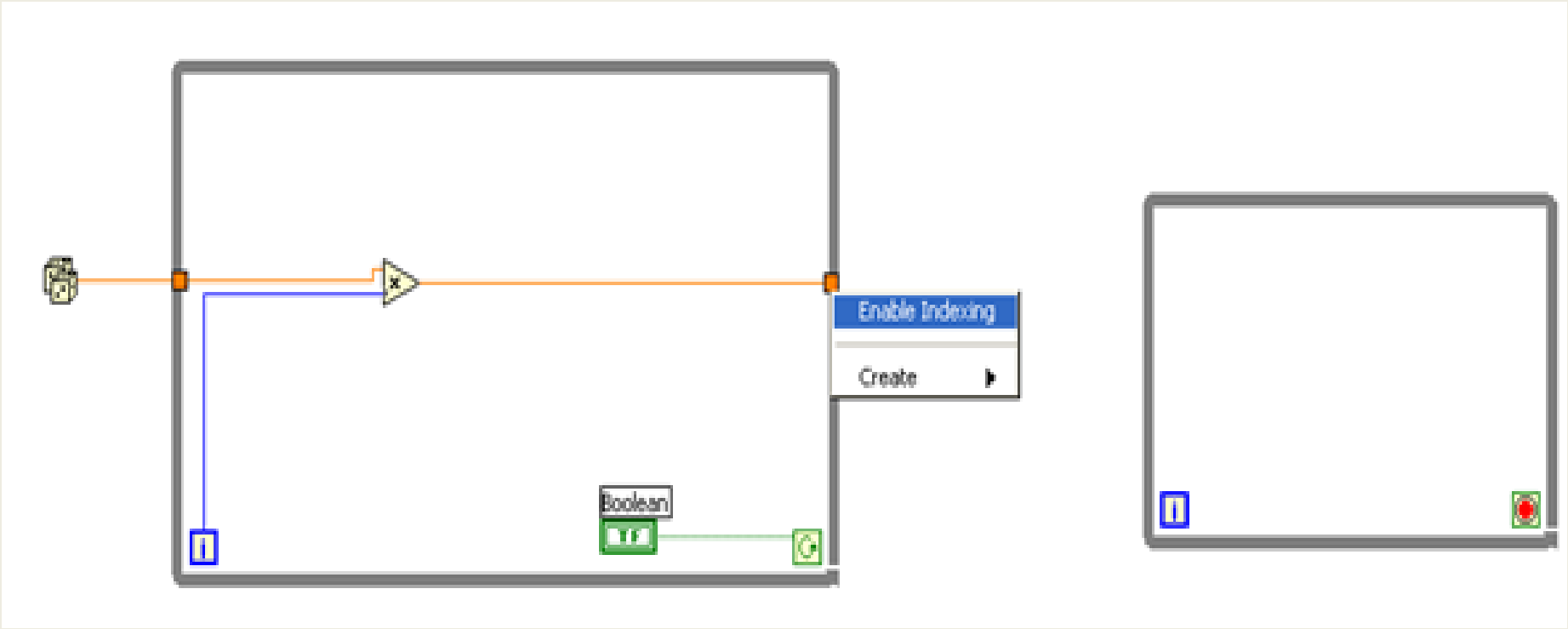
File Edit Operate Tools Browse Window Help

13pt Application Font

Harm. Analyz.

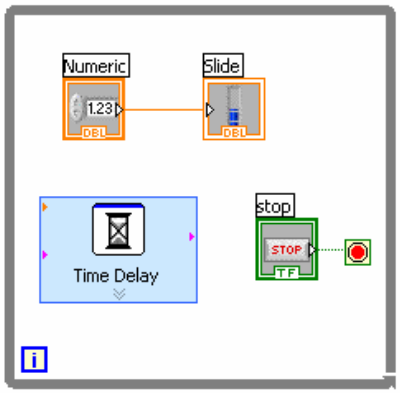




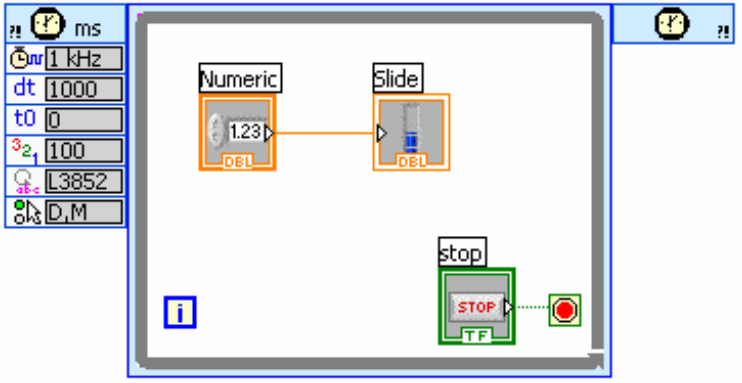

Enable Indexing
Create
Boolean

How Do I Time a Loop?

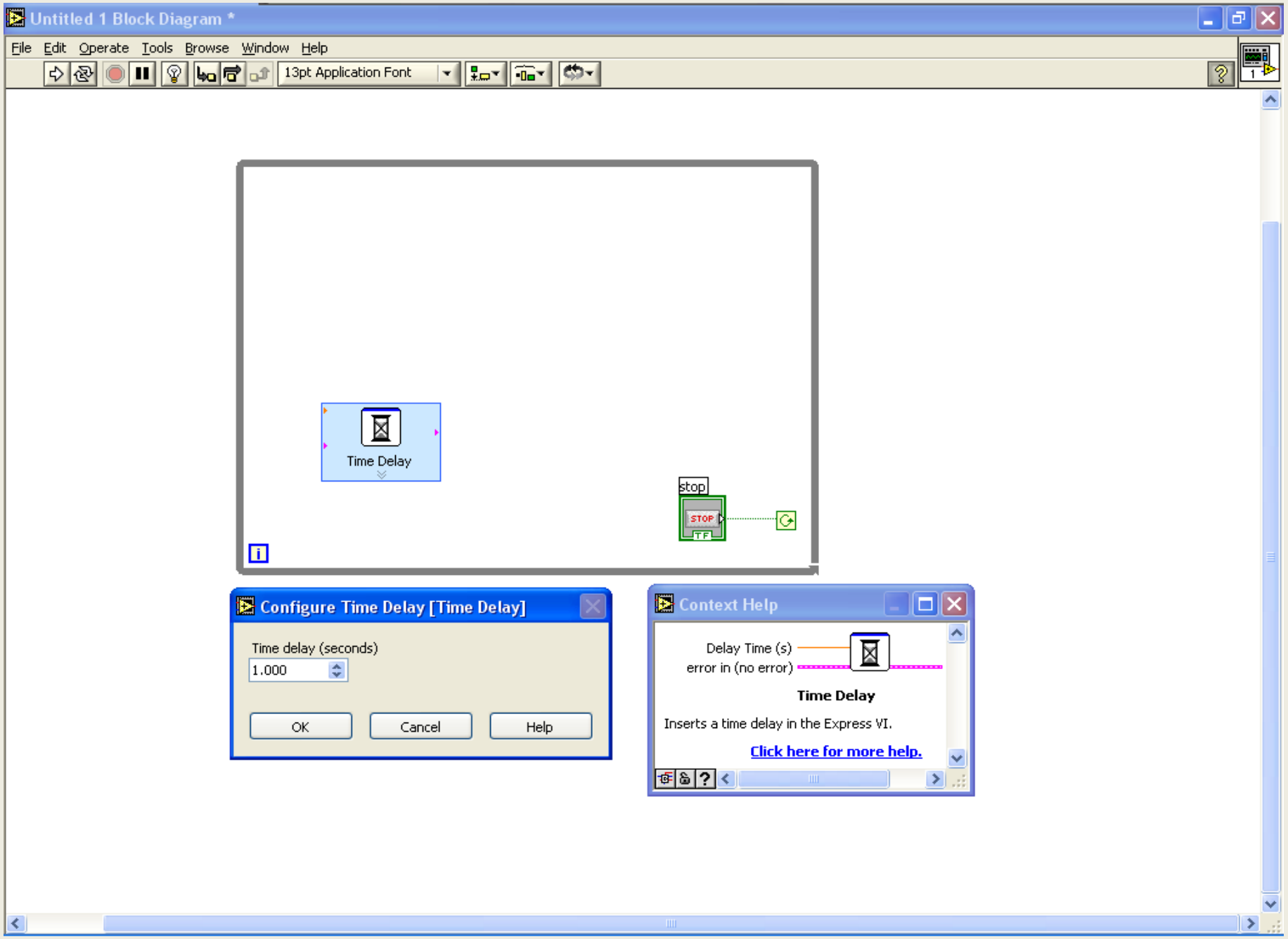
1. Loop Time Delay
   - Configure the Time Delay Express VI for seconds to wait each iteration of the loop (works on For and While loops).
2. Timed Loops
   - Configure special timed While loop for desired *dt.*



Time Delay



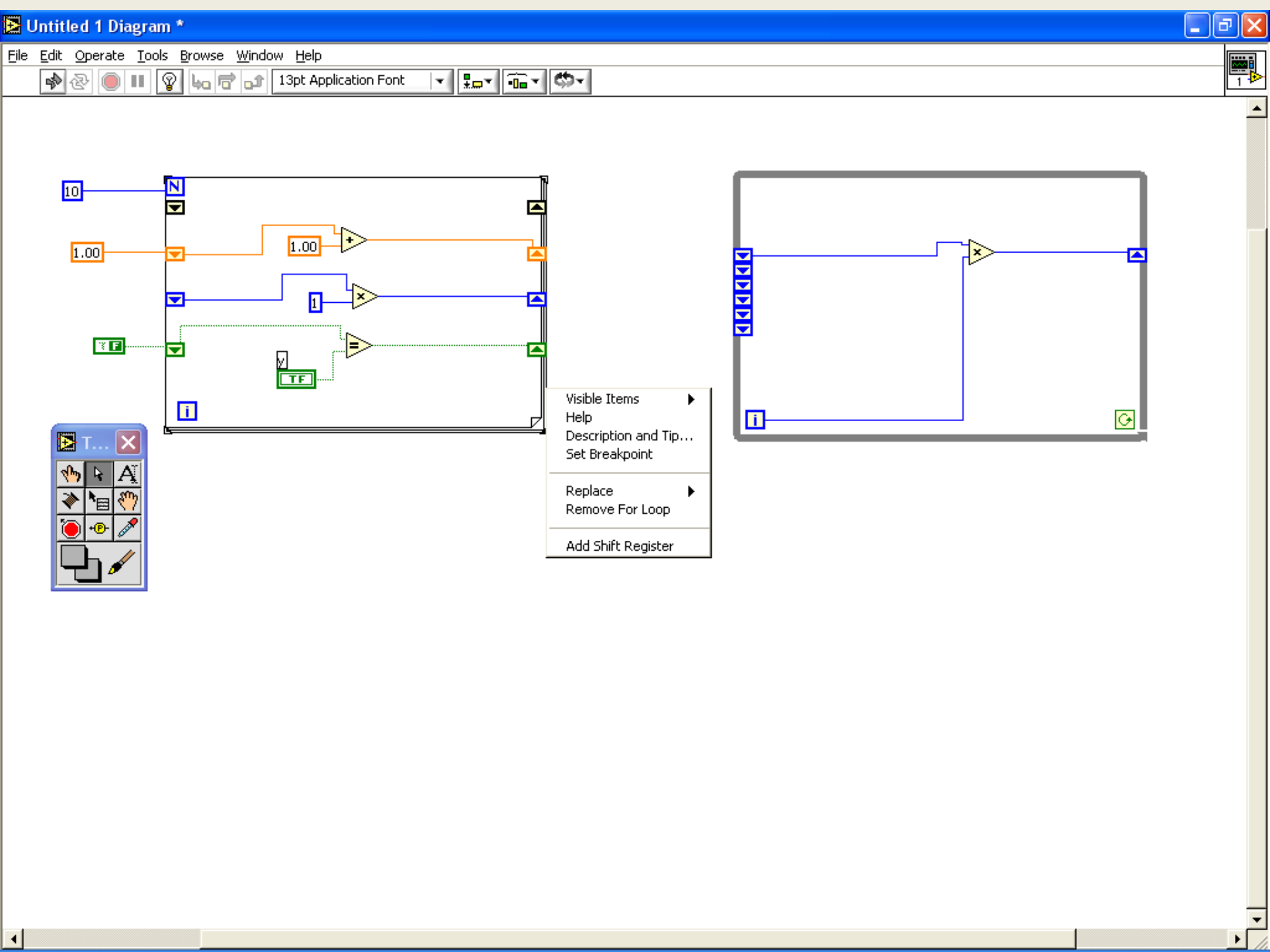
Timed Loop


Untitled 1 Block Diagram *
File Edit Operate Tools Browse Window Help
13pt Application Font
Time Delay
stop
STOP
TF
Configure Time Delay [Time Delay]
Time delay (seconds)
1.000
OK
Cancel
Help
Context Help
Delay Time (s)
error in (no error)
Time Delay
Inserts a time delay in the Express VI.
Click here for more help.

Shift Registers

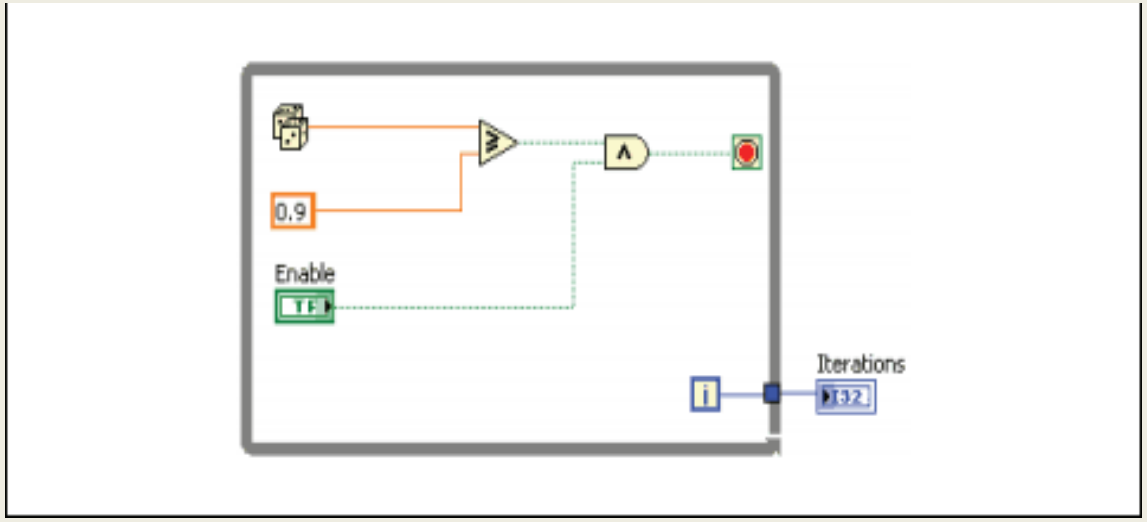
Transfer values from one iteration to the next

Values can be single elements or an array of elements


Untitled 1 Diagram *
File Edit Operate Tools Browse Window Help
13pt Application Font
10
N
1.00
1.00
1
TF
i
Visible Items
Help
Description and Tip...
Set Breakpoint
Replace
Remove For Loop
Add Shift Register
i

Tunnels

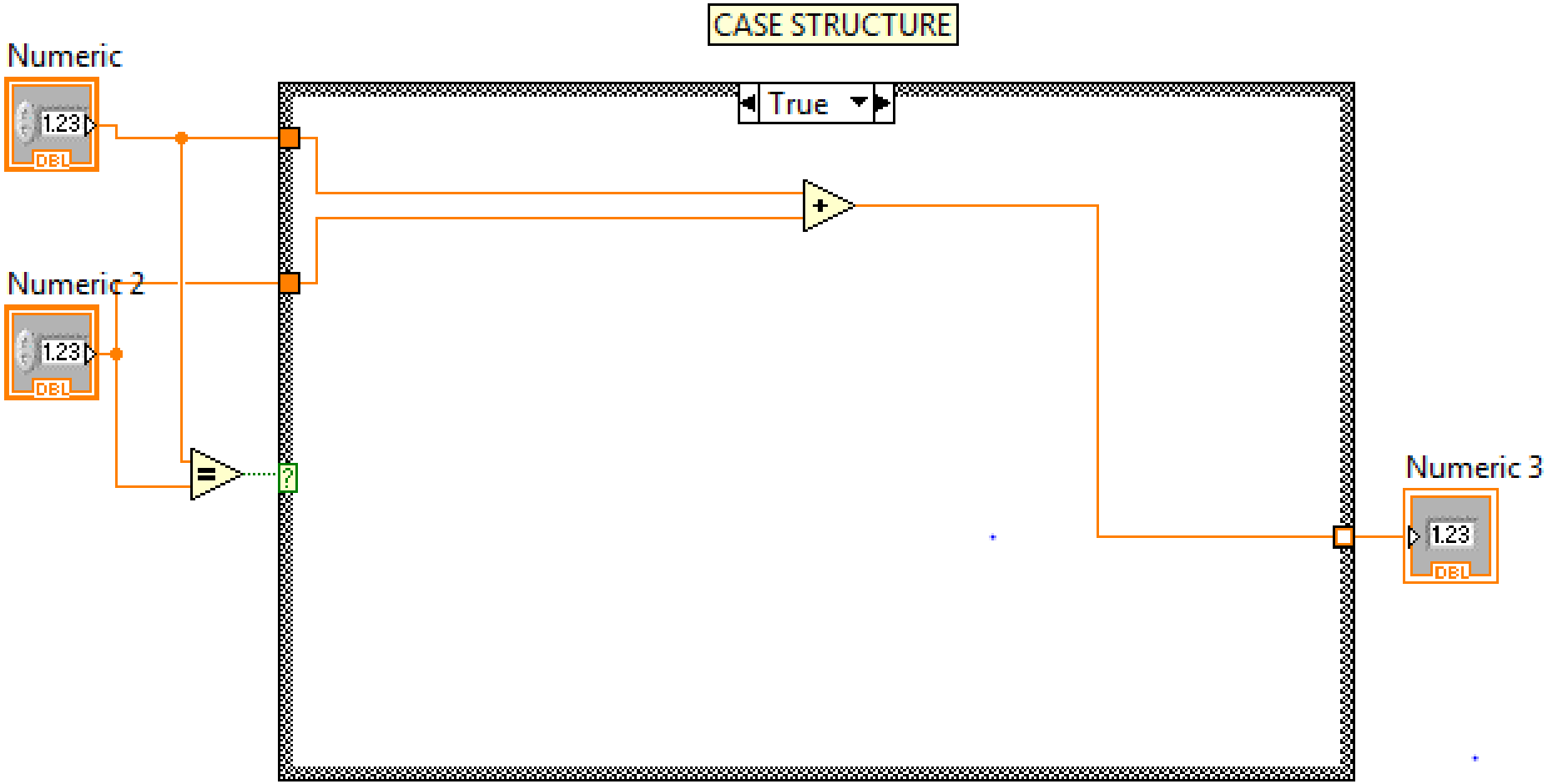
Transfer values from outside to the structure and from loop to outside

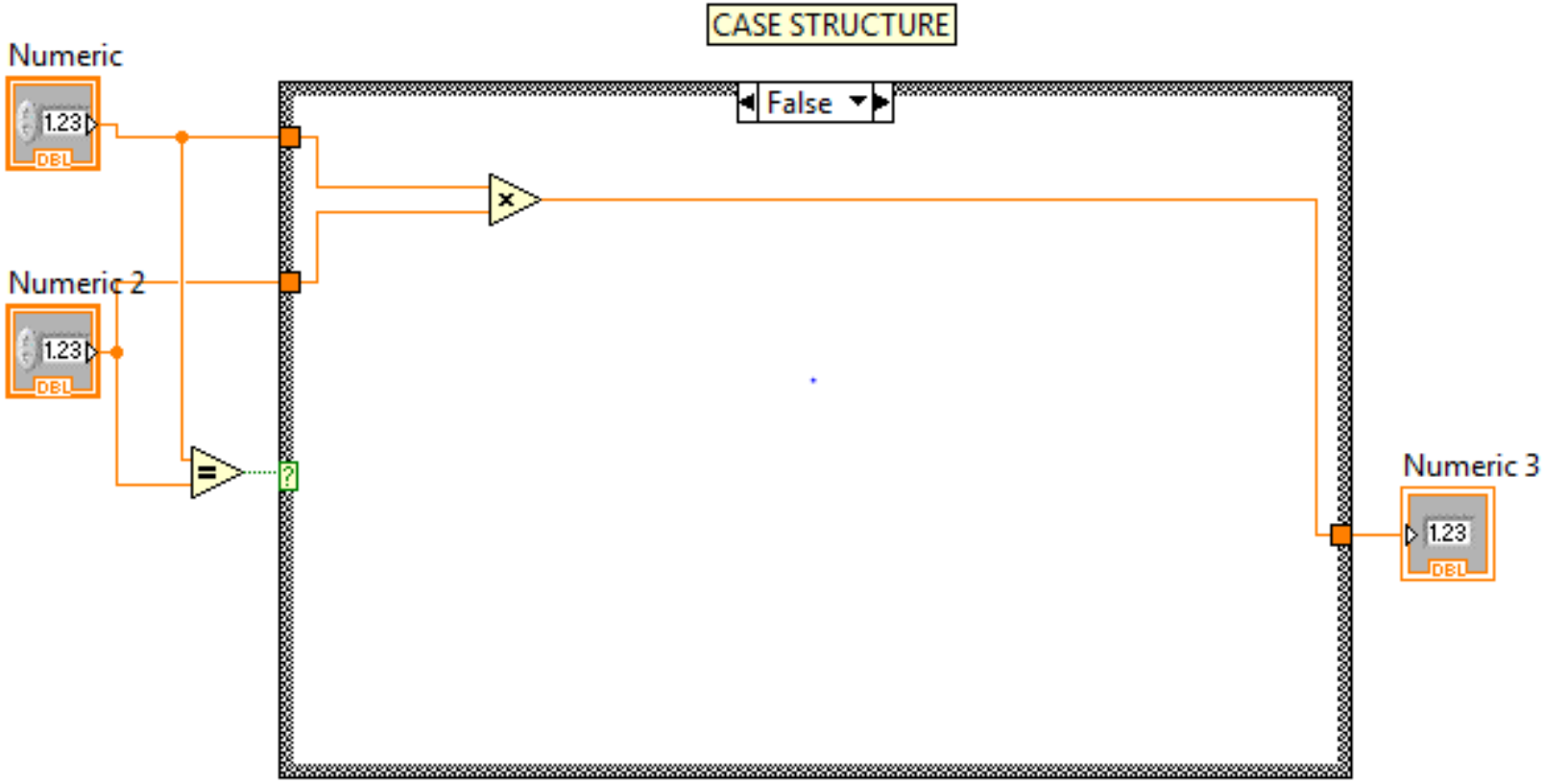


Case Structure

Method of executing conditional text

Text can be Boolean, Numeric, or String

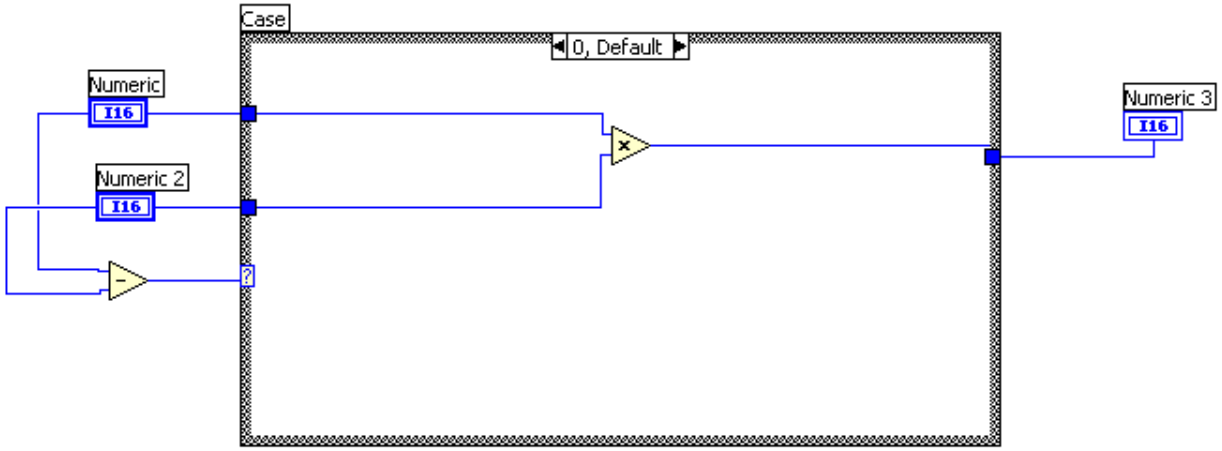

CASE STRUCTURE
Numeric
1.23
DBL
Numeric 2
1.23
DBL
True
?
Numeric 3
1.23
DBL

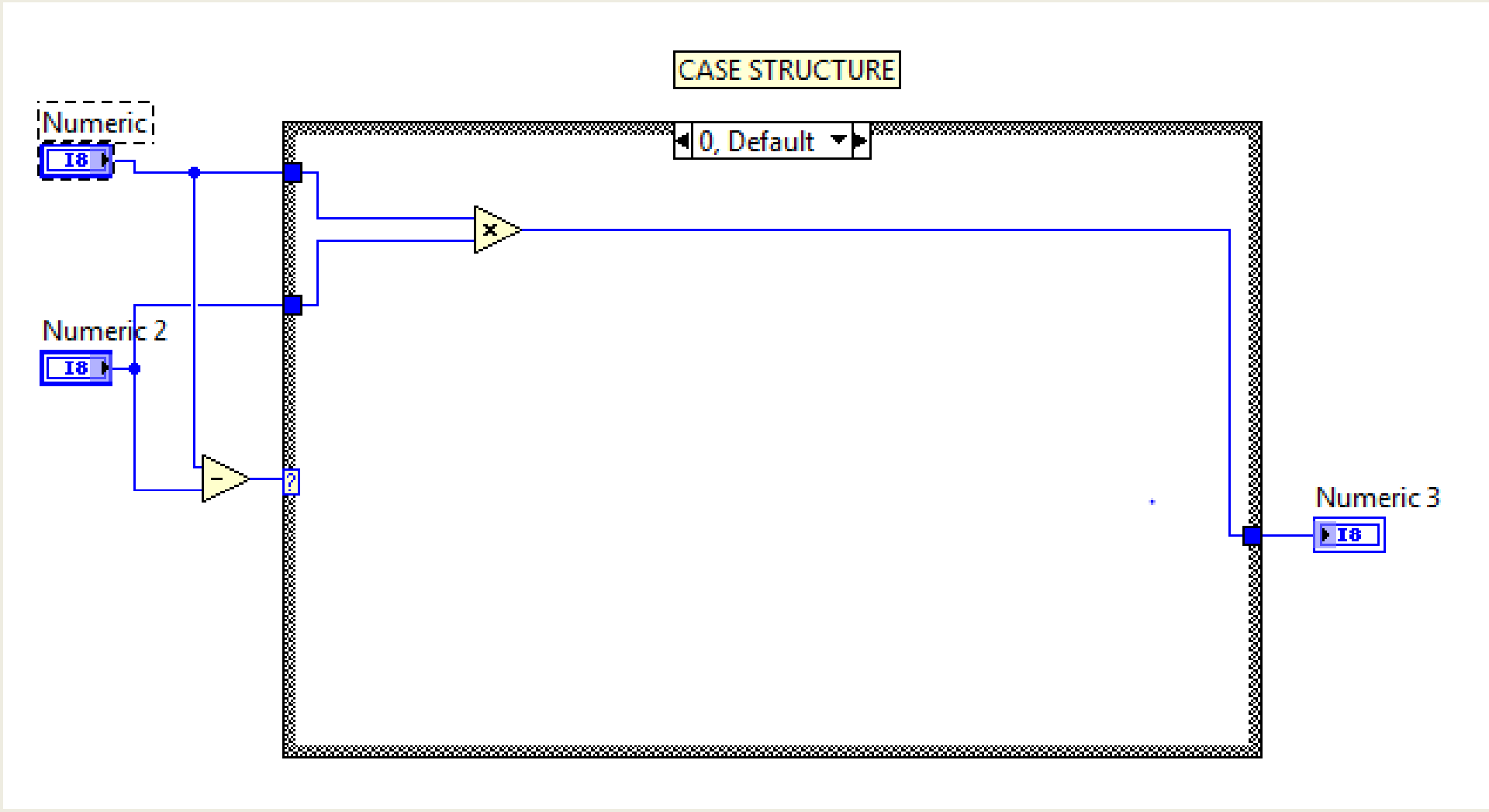

CASE STRUCTURE
Numeric
1.23
DBL
Numeric 2
1.23
DBL
False
×
=
?
Numeric 3
1.23
DBL

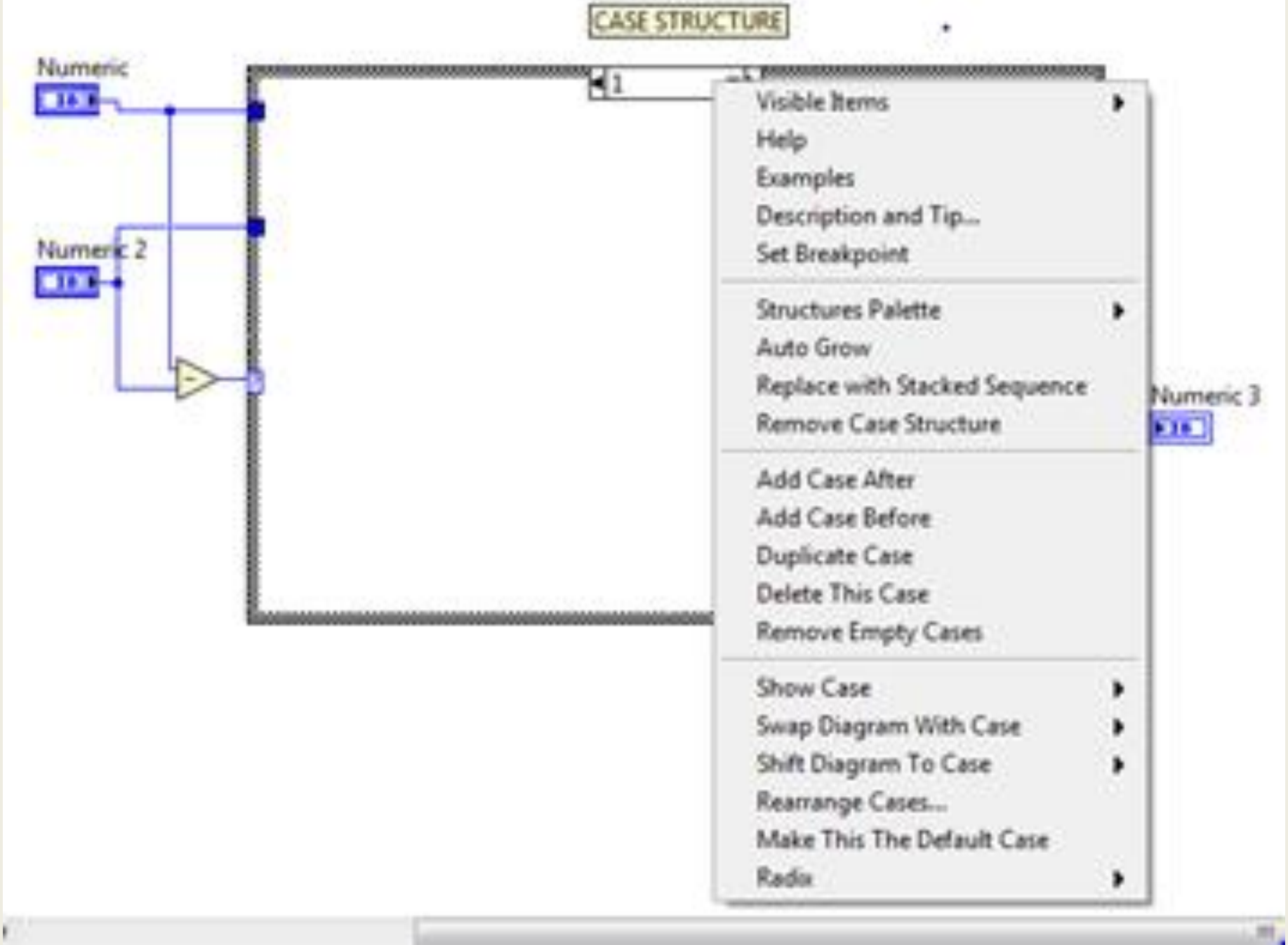
Untitled 2 Diagram *

File Edit Operate Tools Browse Window Help

13pt Application Font



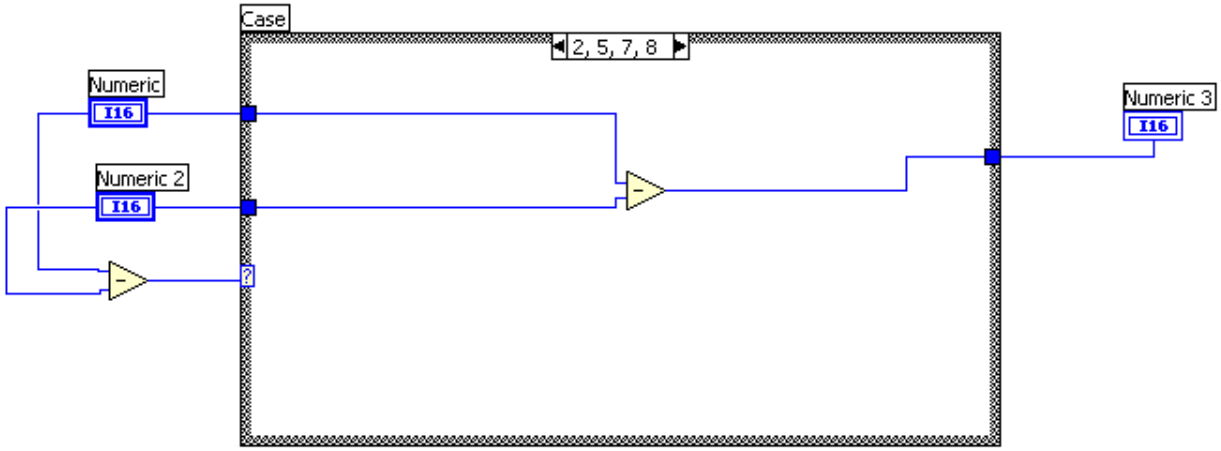

CASE STRUCTURE
Numeric
I8
0, Default
Numeric 2
I8
?
Numeric 3
I8

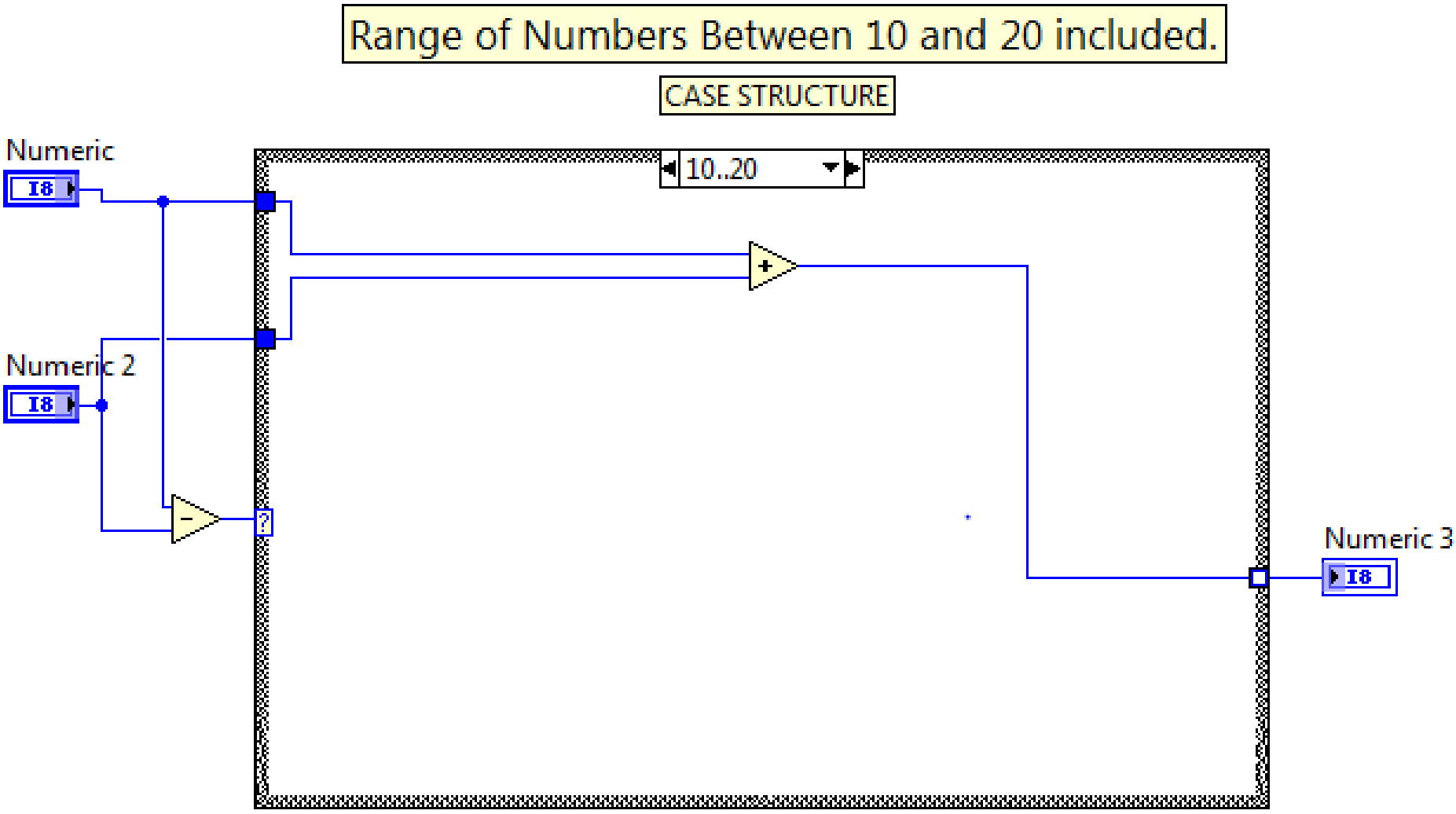

CASE STRUCTURE
Numeric
Numeric 2
1
Visible Items
Help
Examples
Description and Tip...
Set Breakpoint
Structures Palette
Auto Grow
Replace with Stacked Sequence
Remove Case Structure
Add Case After
Add Case Before
Duplicate Case
Delete This Case
Remove Empty Cases
Show Case
Swap Diagram With Case
Shift Diagram To Case
Rearrange Cases...
Make This The Default Case
Radix
Numeric 3

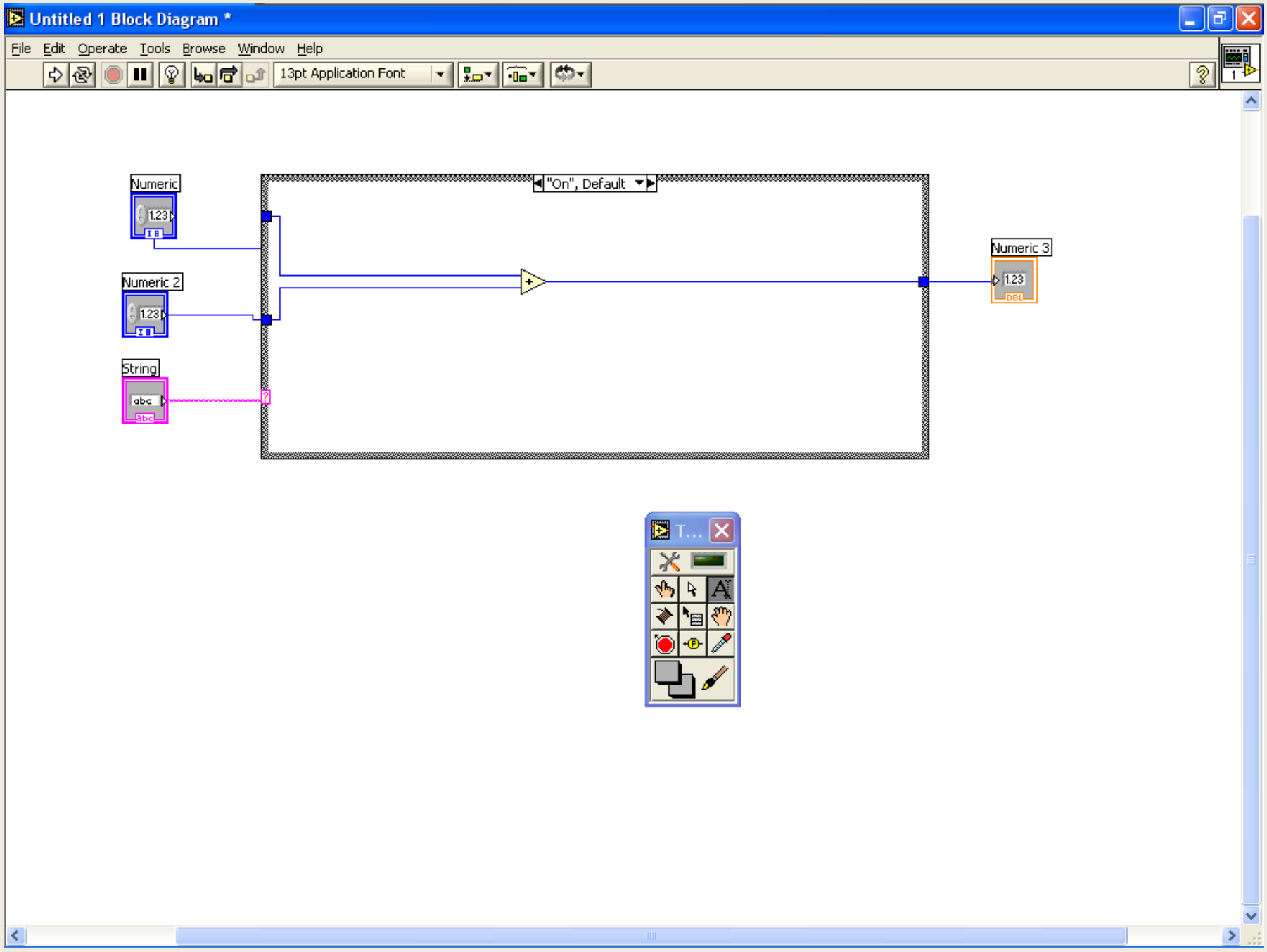
Untitled 2 Diagram *

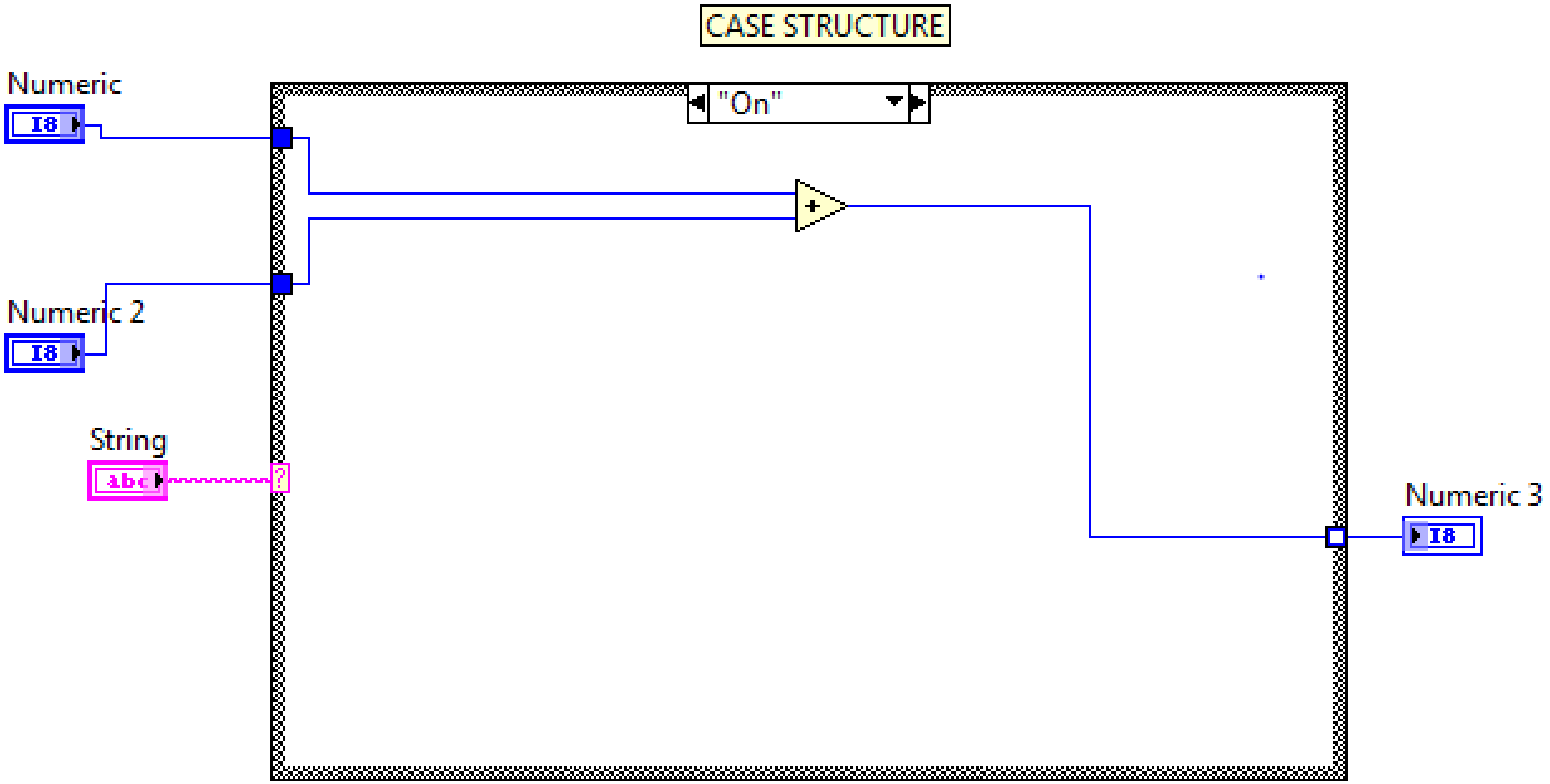
File Edit Operate Tools Browse Window Help

13pt Application Font



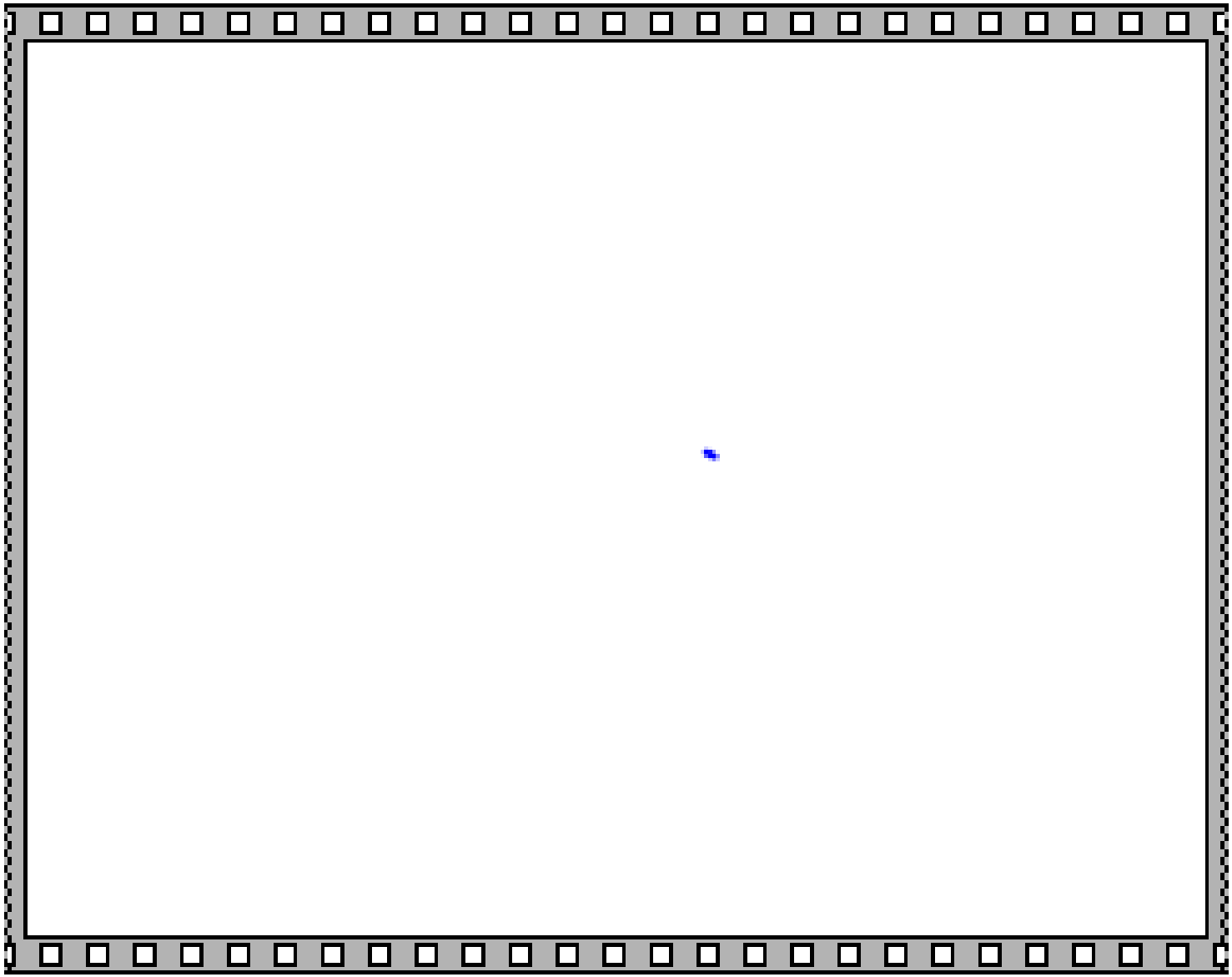

Range of Numbers Between 10 and 20 included.
CASE STRUCTURE
Numeric
I8
Numeric 2
I8
10..20
+
-
?
Numeric 3
I8

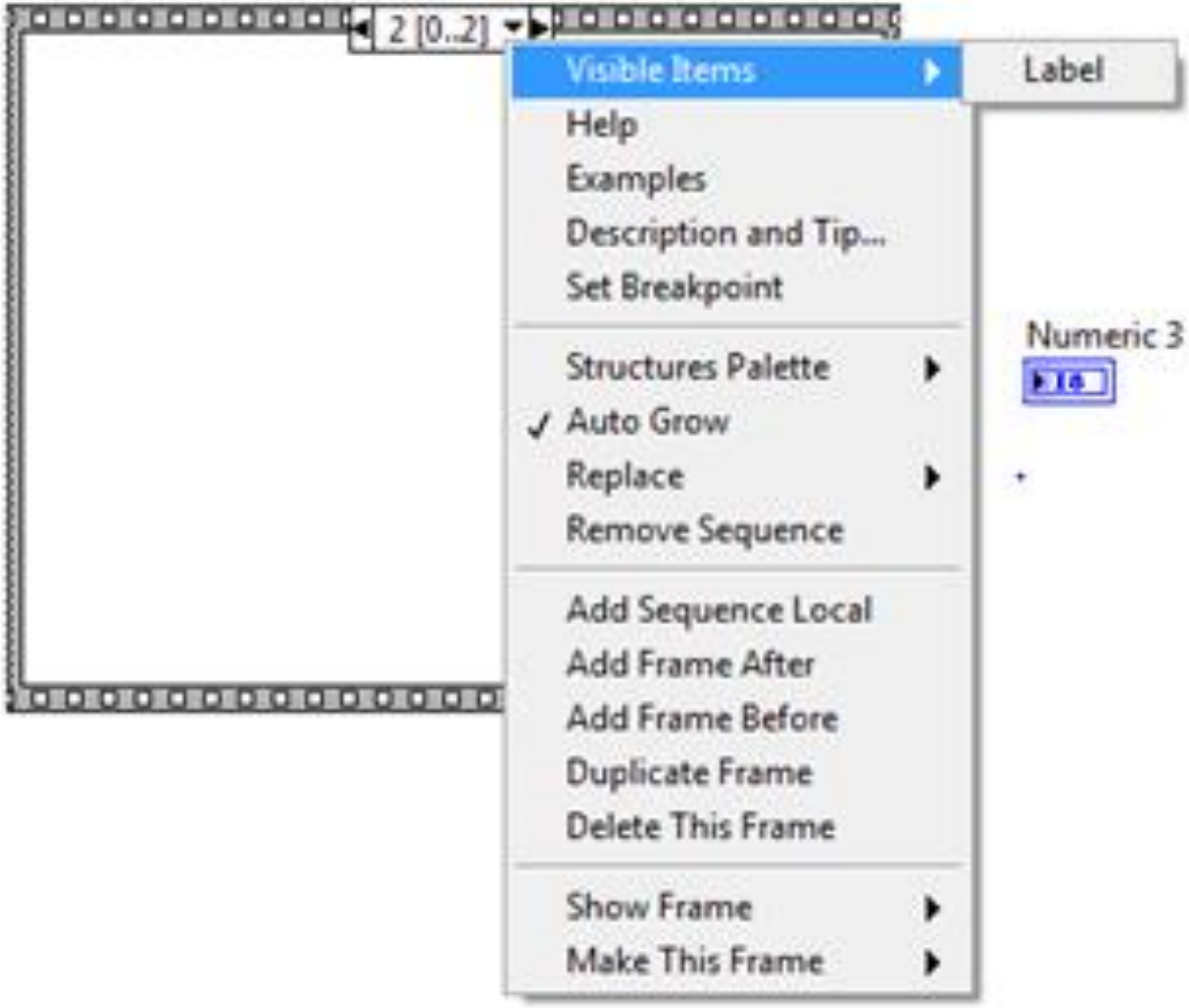

Untitled 1 Block Diagram *
File Edit Operate Tools Browse Window Help
13pt Application Font
"On", Default
Numeric
Numeric 2
String
Numeric 3
T...

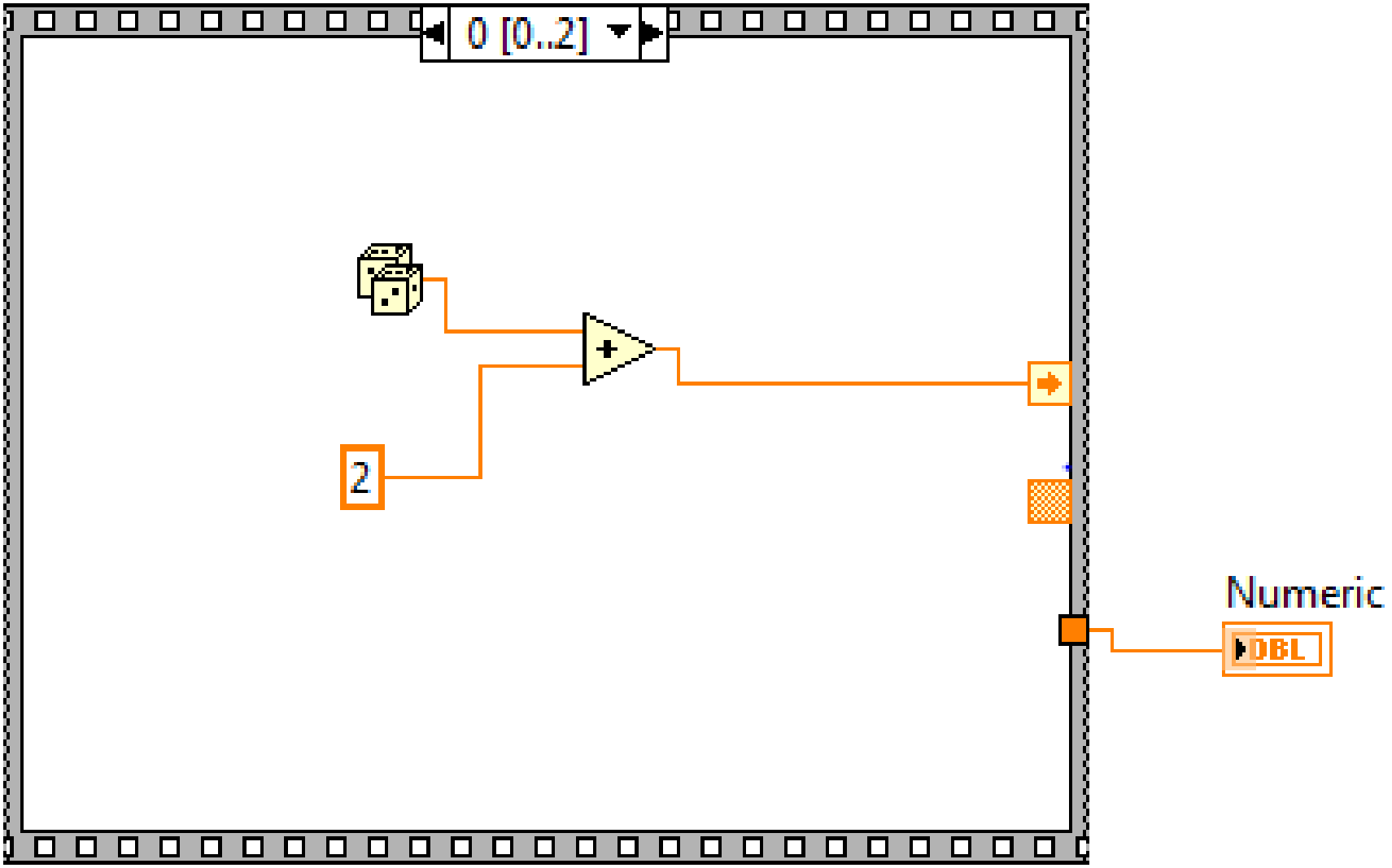

CASE STRUCTURE
Numeric
I8
"On"
Numeric 2
I8
String
abc
?
Numeric 3
I8

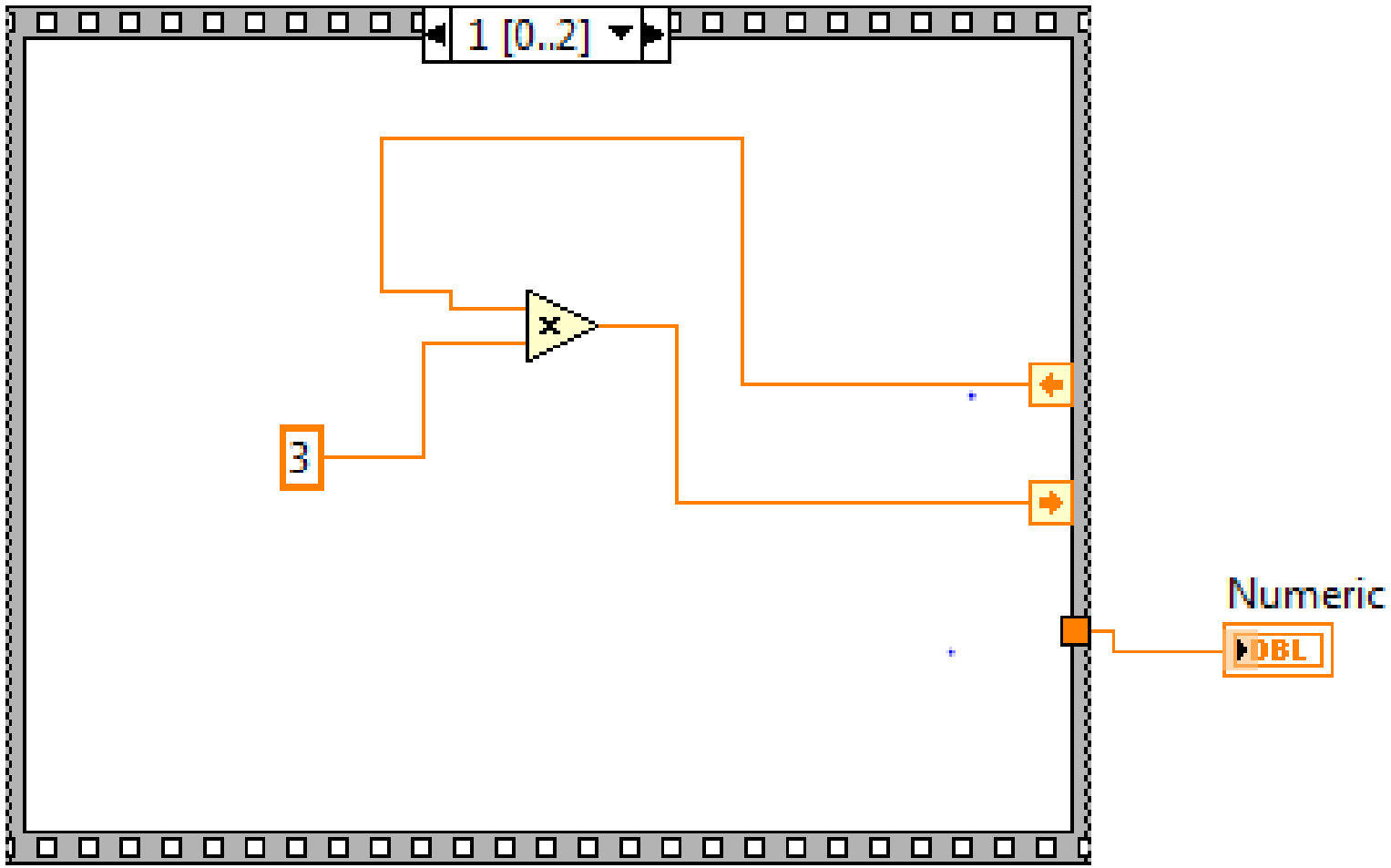
Sequence Structure

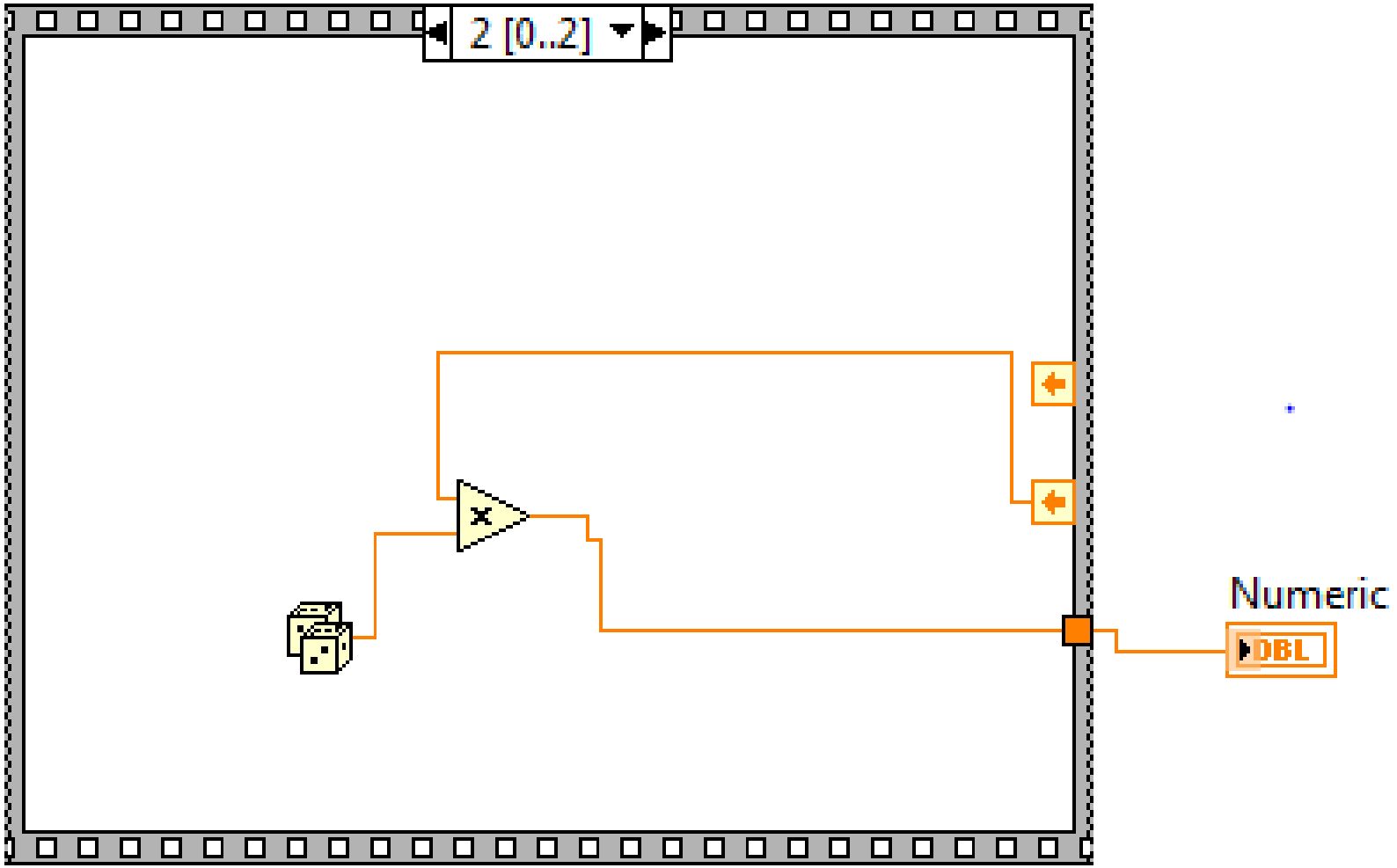
> Executes sub-diagrams sequentially

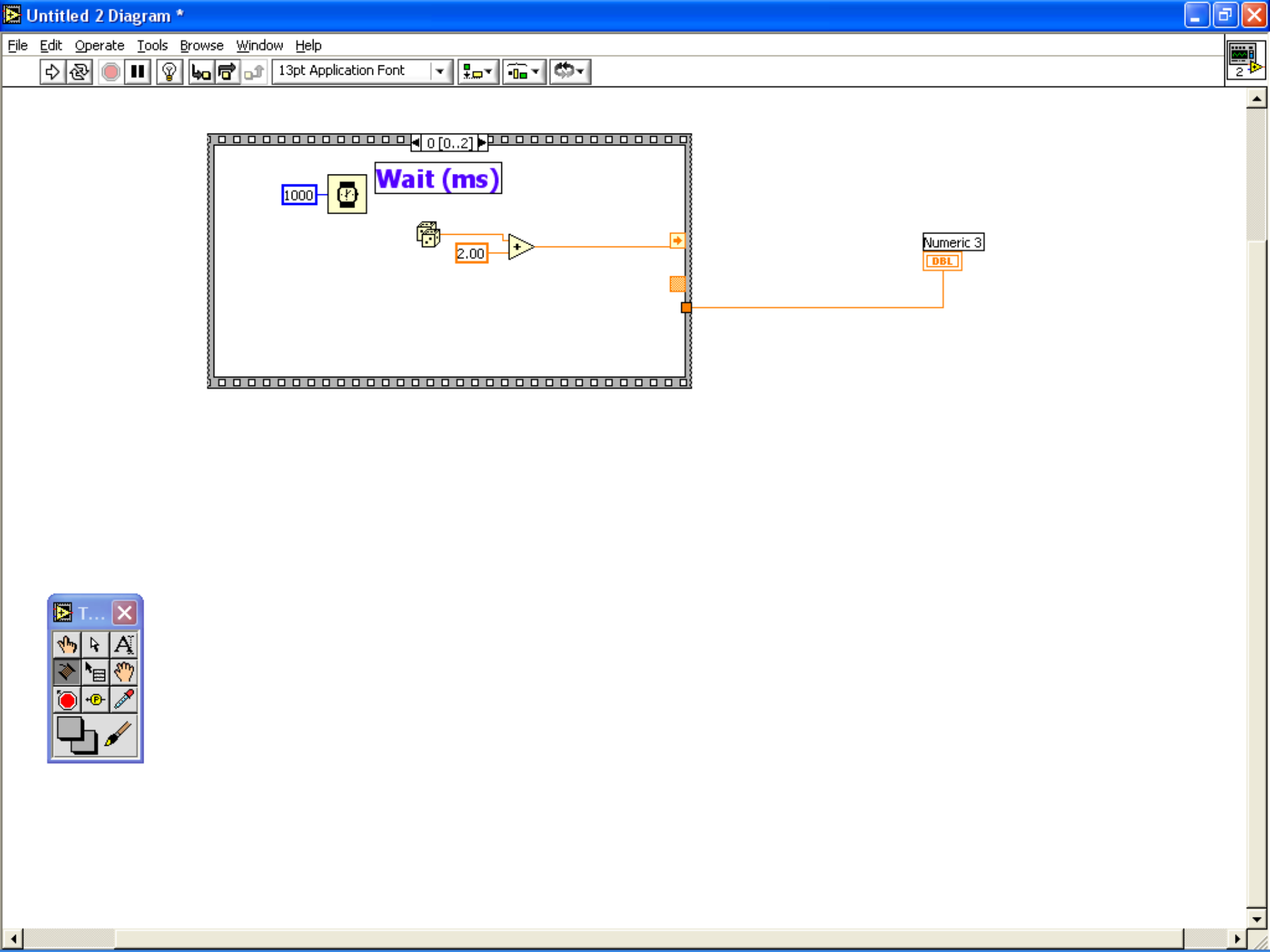



2 [0..2]
Visible Items
Label
Help
Examples
Description and Tip...
Set Breakpoint
Structures Palette
Auto Grow
Replace
Remove Sequence
Add Sequence Local
Add Frame After
Add Frame Before
Duplicate Frame
Delete This Frame
Show Frame
Make This Frame
Numeric 3


0 [0..2]
2
Numeric
DBL


1 [0..2]
x
3
Numeric
DBL

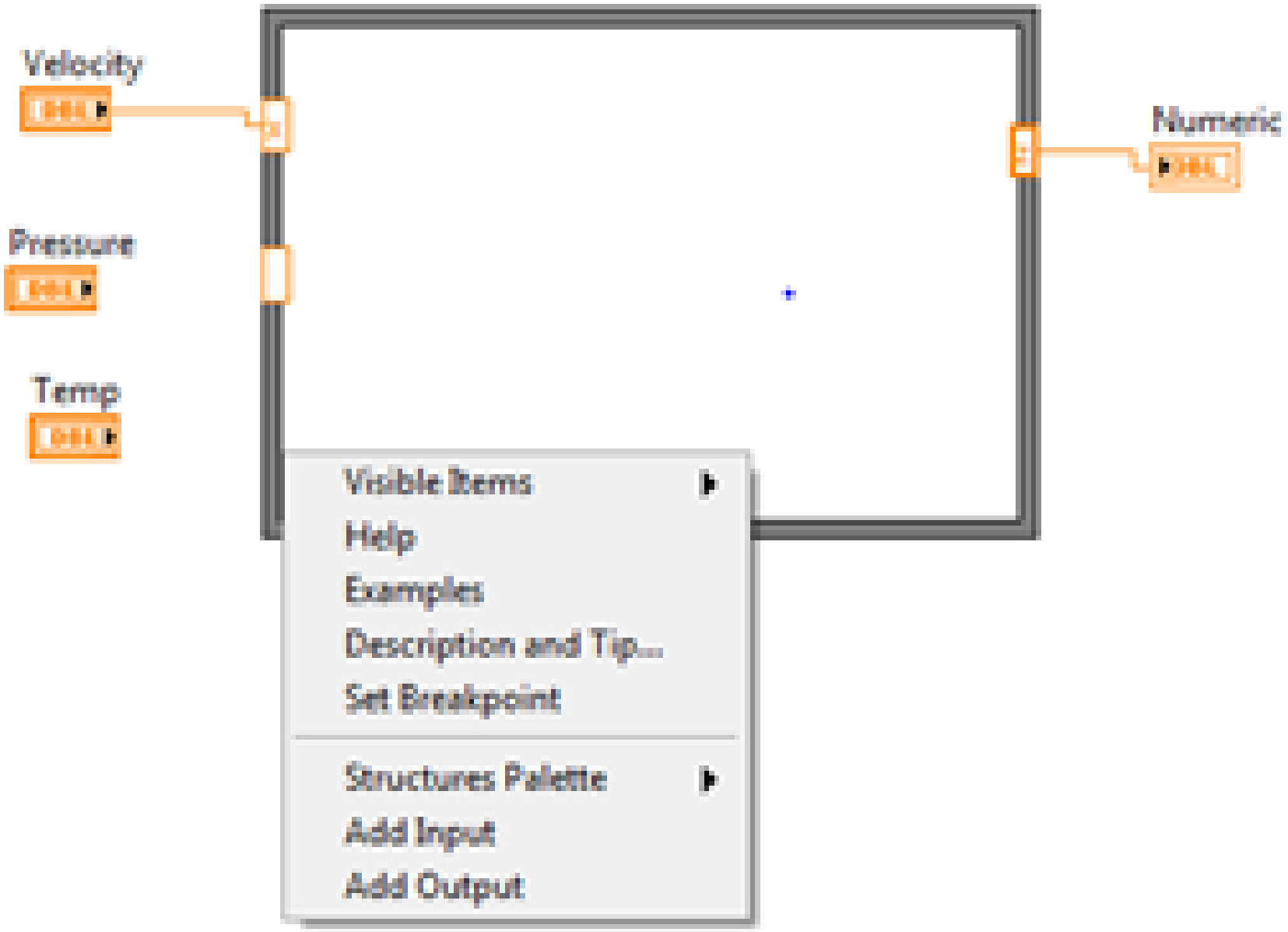

2 [0..2]
Numeric
DBL


Untitled 2 Diagram *
File Edit Operate Tools Browse Window Help
13pt Application Font
0 [0..2]
Wait (ms)
1000
2.00
Numeric 3
DBL

Formula Node

Allows programming of one or more algebraic formulas

Use Syntax similar to text-based programming languages


Velocity
Pressure
Temp
Numeric
Visible Items
Help
Examples
Description and Tip...
Set Breakpoint
Structures Palette
Add Input
Add Output

Untitled 3 Diagram *

File Edit Operate Tools Browse Window Help

13pt Application Font

```
t=(x*y)*exp(z);
s=x+z*sin(y);
```

Temp DBL x

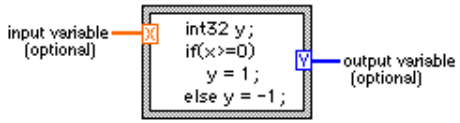
Pressure DBL y

Velocity DBL z

t Numeric DBL

s Numeric 2 DBL

Context Help



Formula Node

Note: Starting in LabVIEW 6.0, the ^ operator is no longer the power operator. Instead, it now means bit-wise exclusive OR. Use the ** operator for the power operator.

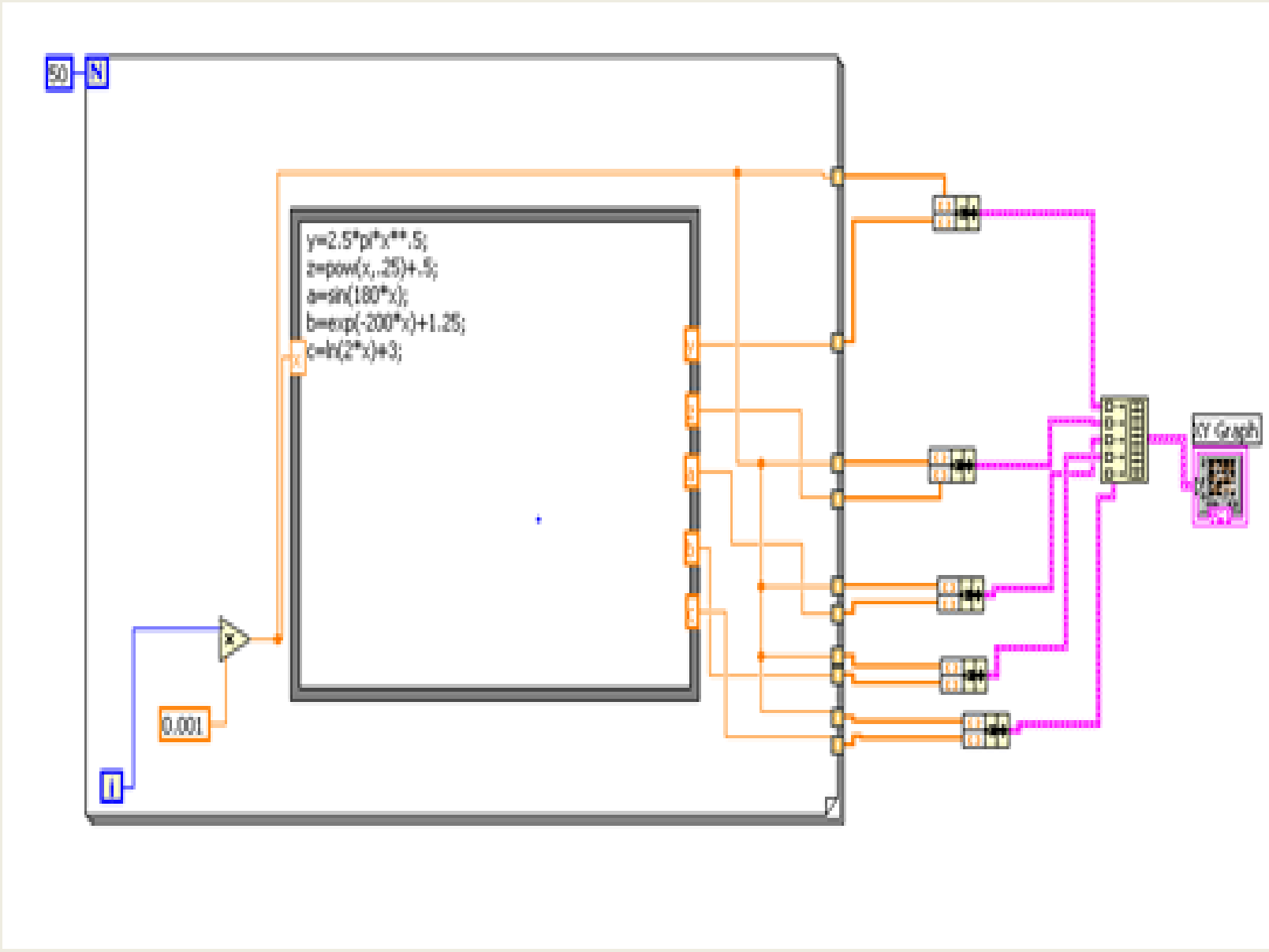
The formula node is a resizable box that contains one or more text-based formula statements delimited by a semicolon, as in the following example:

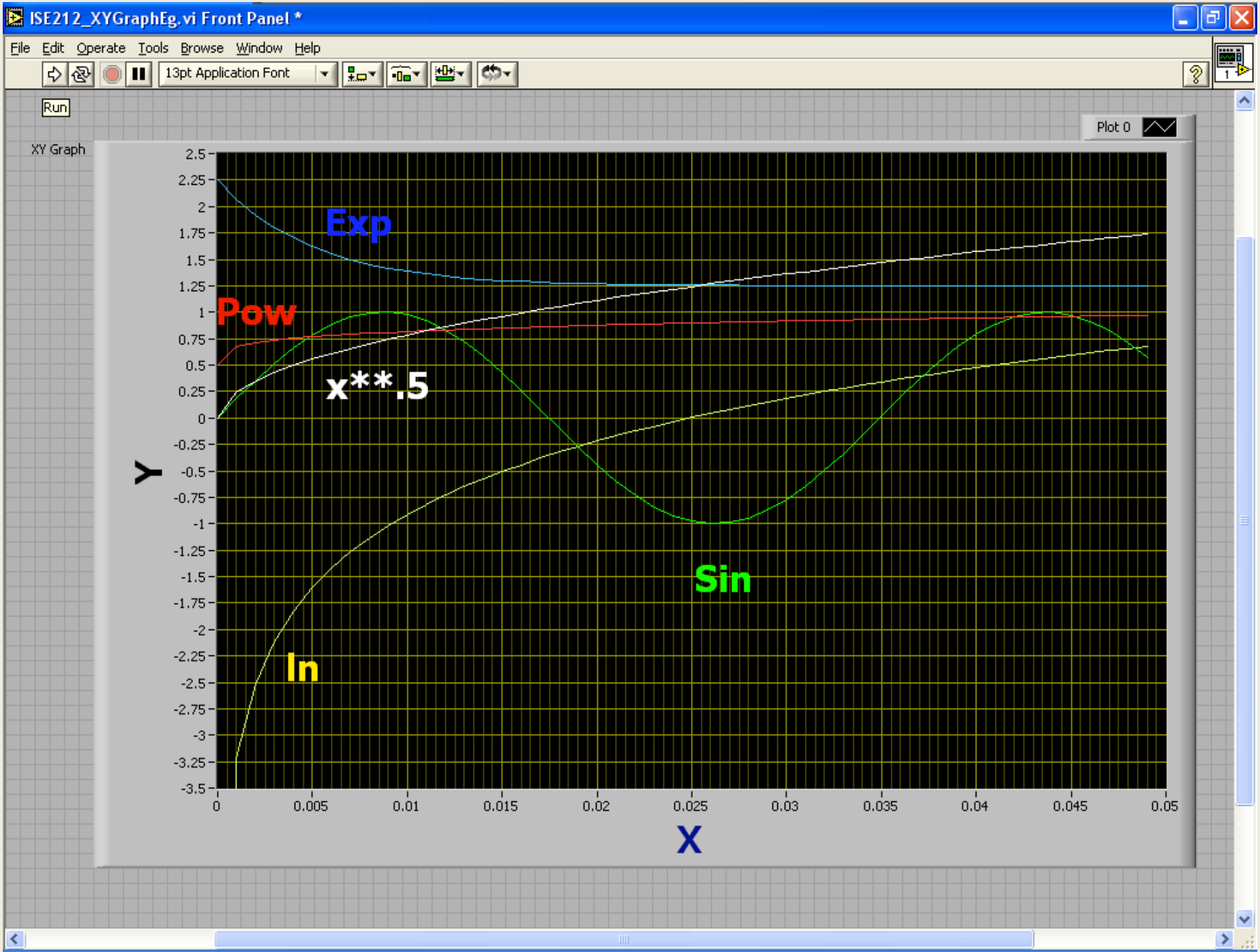
y=3*x-2+x*log(x);

Formula statements use a syntax similar to most text-based programming languages for arithmetic expressions. You can add comments by enclosing them inside a slash-asterisk pair (/* comment */).

The pop-up menu on the border of the Formula Node contains items to add input and output variables. Output variables are distinguished by a thicker border. No two inputs and no two outputs can have the same name. However, an output can have the same name as an input. All variables are floating point numeric scalars.

If a syntax error occurs, you can click the broken Run button to get the error


50
N
y=2.5*pi*x**.5;
z=pow(x,.25)+.5;
a=sin(180*x);
b=exp(-200*x)+1.25;
c=ln(2*x)+3;
x
y
z
a
b
c
0.001
i
XY Graph


ISE212_XYGraphEg.vi Front Panel *
File Edit Operate Tools Browse Window Help
13pt Application Font
Run
XY Graph
Plot 0
Exp
Pow
x**.5
Sin
ln
Y
X
2.5
2.25
2
1.75
1.5
1.25
1
0.75
0.5
0.25
0
-0.25
-0.5
-0.75
-1
-1.25
-1.5
-1.75
-2
-2.25
-2.5
-2.75
-3
-3.25
-3.5
0
0.005
0.01
0.015
0.02
0.025
0.03
0.035
0.04
0.045
0.05

UML
First Lecture

A Very Brief Introduction to UML

Overview

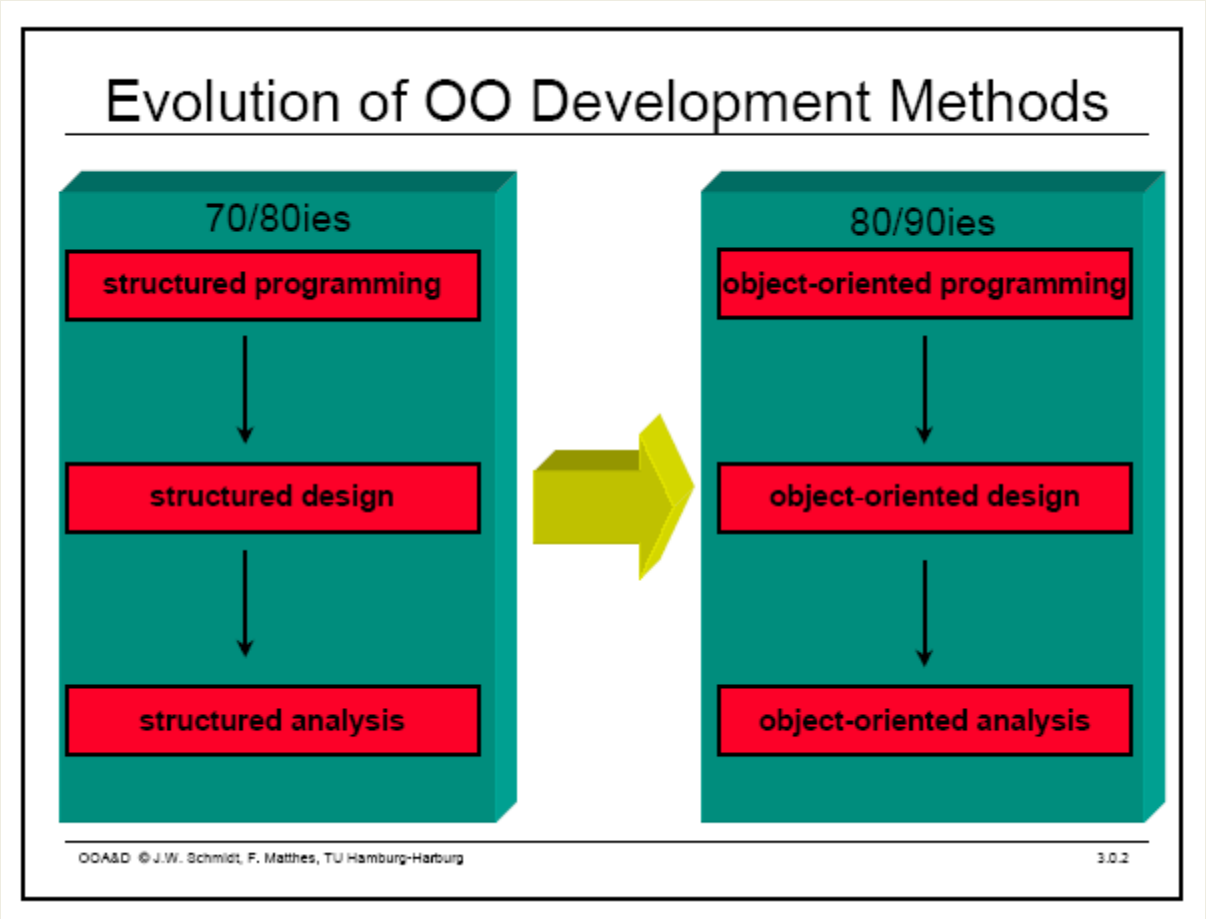
- What is Modeling?
- What is UML?
- A brief history of UML
- Understanding the basics of UML
- UML diagrams
- UML Modeling tools

Modeling

- Describing a system at a high level of abstraction
  - A model of the system
  - Used for requirements and specifications

- Is it necessary to model software systems?

Object Oriented Modeling



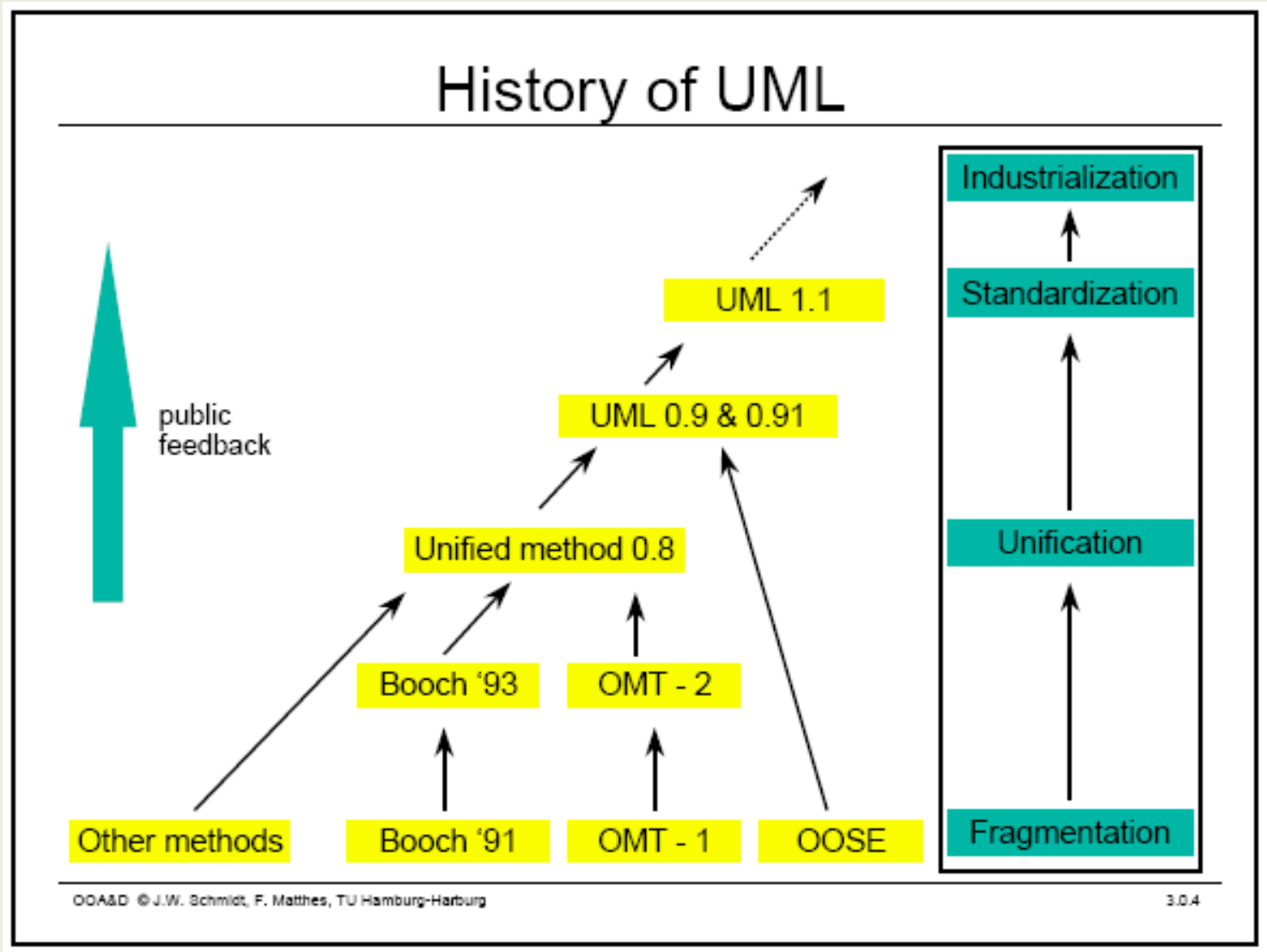
What is UML?

- UML stands for “Unified Modeling Language”
- It is a industry-standard graphical language for specifying, visualizing, constructing, and documenting the artifacts of software systems
- The UML uses mostly graphical notations to express the OO analysis and design of software projects.
- Simplifies the complex process of software design

Why UML for Modeling

- Use graphical notation to communicate more clearly than natural language (imprecise) and code(too detailed).
- Help acquire an overall view of a system.
- UML is *not* dependent on any one language or technology.
- UML moves us from fragmentation to standardization.

History of UML



2005

1997

Two Broad Categories of UML Diagrams

- Structural Diagrams
- Behavioral Diagrams

Structural Diagrams

- These Diagrams represent the static aspects of a system. Further types of structural Diagrams

  - **Class diagram**
  - **Object diagram**
  - **Component diagram**
  - **Deployment diagram**

Behavioral Diagrams

- These diagrams represent the dynamic aspects of a system. Further types of behavioral diagrams
  - **Use case diagram**
  - **Sequence diagram**
  - **Collaboration diagram**
  - **Statechart diagram**
  - **Activity diagram**

Types of UML Diagrams Discussed in this Course

- **Use Case Diagram**
- **Class Diagram**
- **Sequence Diagram**
- **State Diagram**
- **Activity Diagram**

This is only a subset of diagrams … but are most widely used

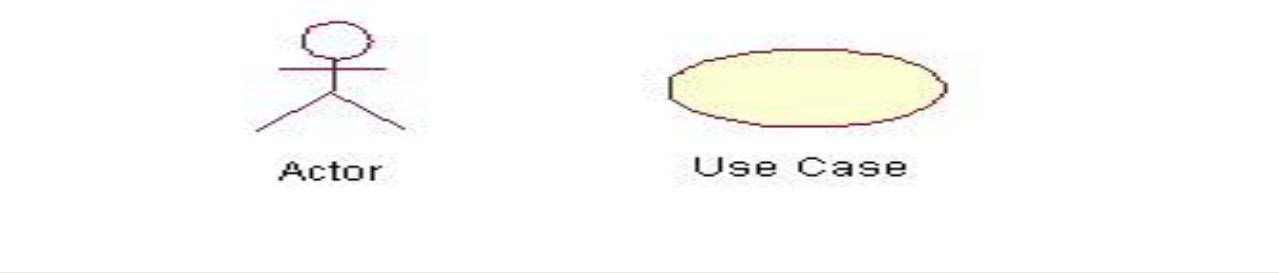
Use Case Diagram

- Used for describing a set of user **scenarios**
- Mainly used for capturing user requirements
- Work like a **contract** between end user and software developers

Use Case Diagram (core components)

Actors: A role that a user plays with respect to the system,including human users and other systems. e.g.,inanimate physical objects (e.g. robot); an external system that needs some information from the current system.

Use case: A task to be accomplished.

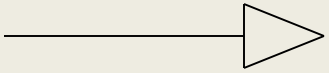


System boundary: rectangle diagram representing the boundary between the actors and the system.

Use Case Diagram(core relationship)

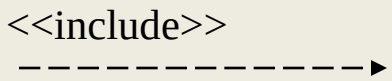
Association: communication between an actor and a use case; Represented by a solid line.

Generalization: relationship between one general use case and a special use case (used for defining special alternatives)
Represented by a line with a triangular arrow head toward the parent use case.

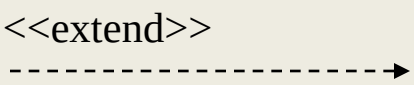


Use Case Diagram(core relationship)

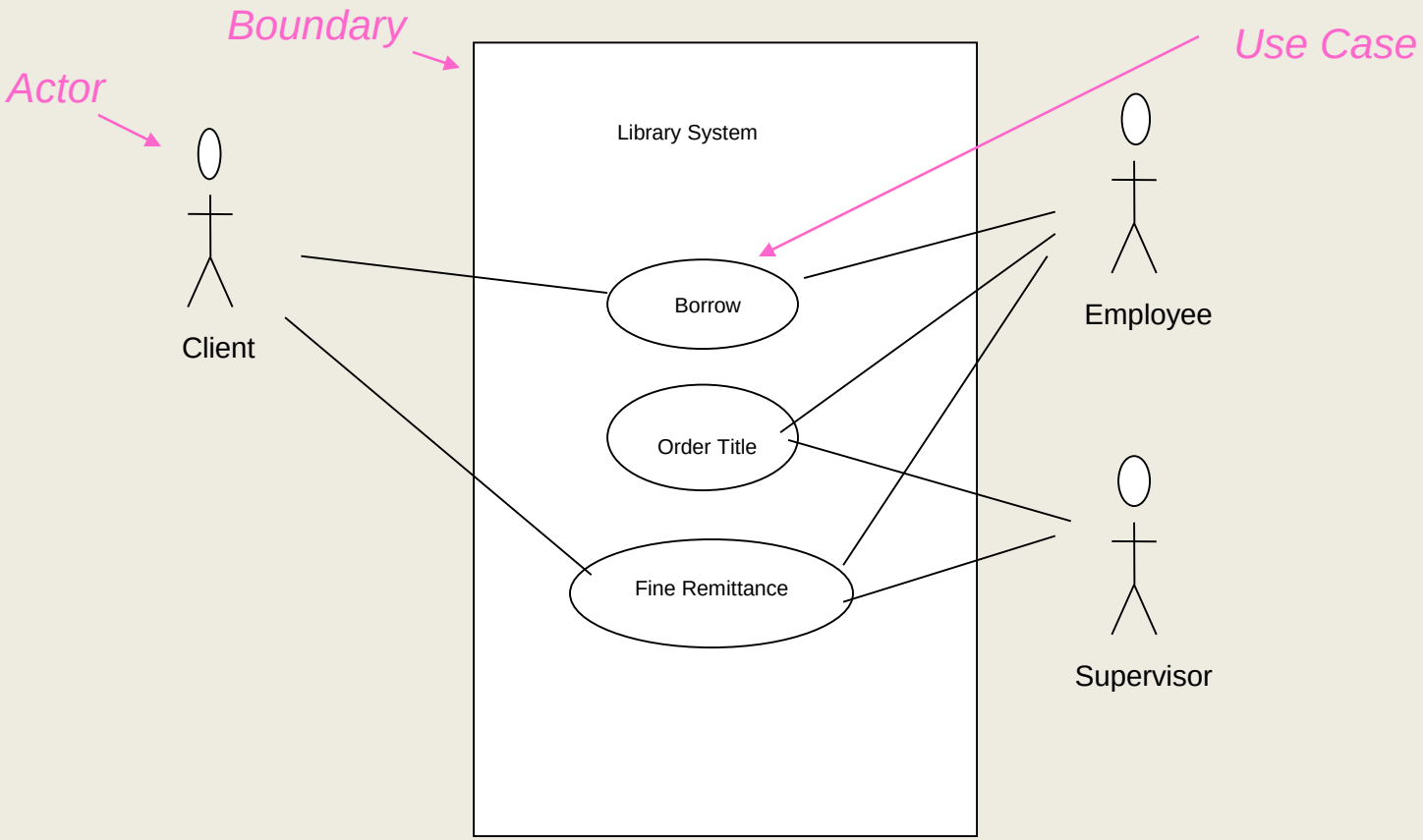
Include: a dotted line labeled <<include>> beginning at base use case and ending with an arrows pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use "include" in stead of copying the description of that behavior.



Extend: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares "extension points".

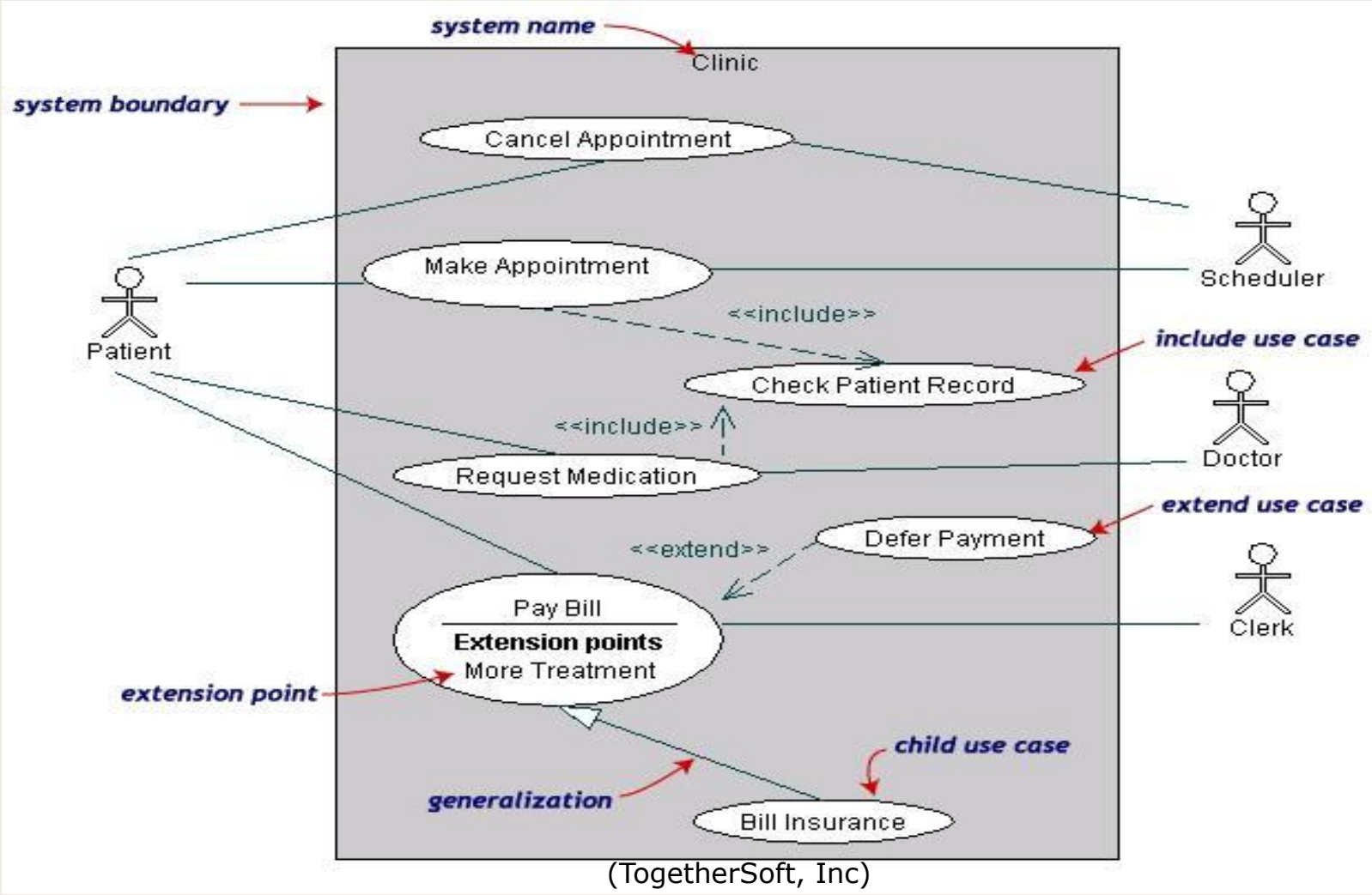


Use Case Diagrams



- A generalized description of how a system will be used.
- Provides an overview of the intended functionality of the system

Use Case Diagrams(cont.)



(TogetherSoft, Inc)

Use Case Diagrams(cont.)

•**Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)

•Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask.(include)

•The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)

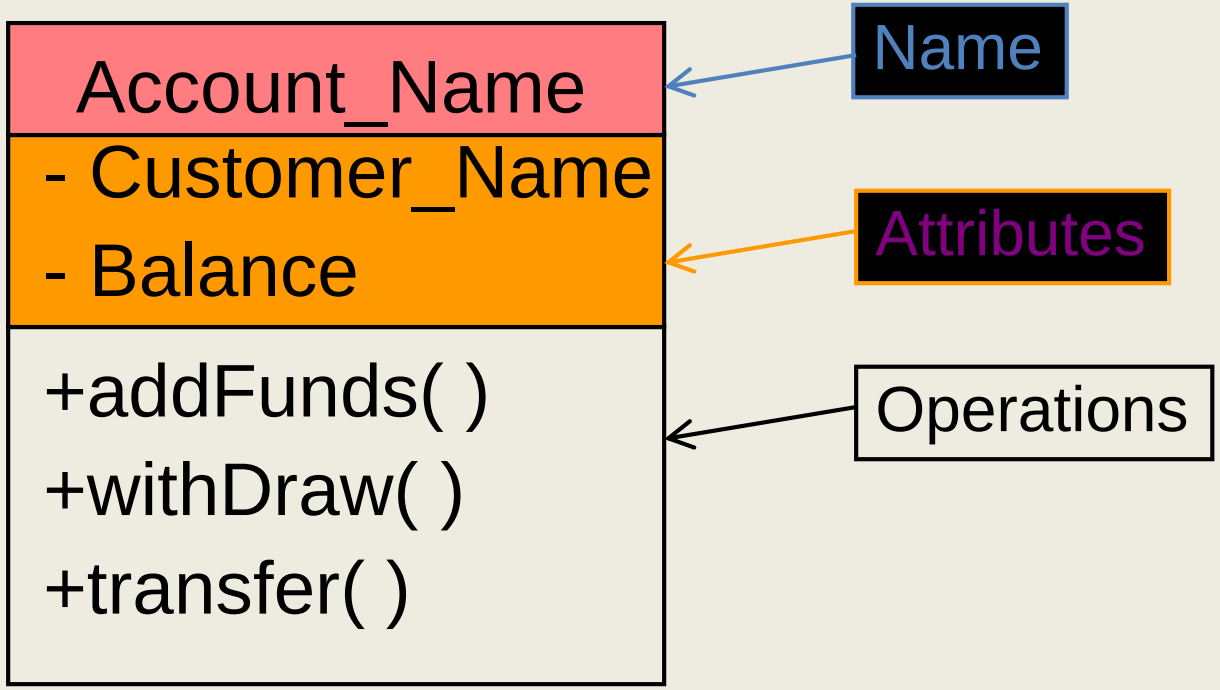
Class diagram

- Used for describing structure and behavior in the use cases
- Provide a conceptual model of the system in terms of entities and their relationships
- Used for requirement capture, end-user interaction
- Detailed class diagrams are used for developers

Class representation

- Each class is represented by a rectangle subdivided into three compartments
  - Name
  - Attributes
  - Operations
- Modifiers are used to indicate visibility of attributes and operations.
  - ‘+’ is used to denote *Public* visibility (everyone)
  - ‘#’ is used to denote *Protected* visibility (friends and derived)
  - ‘-’ is used to denote *Private* visibility (no one)

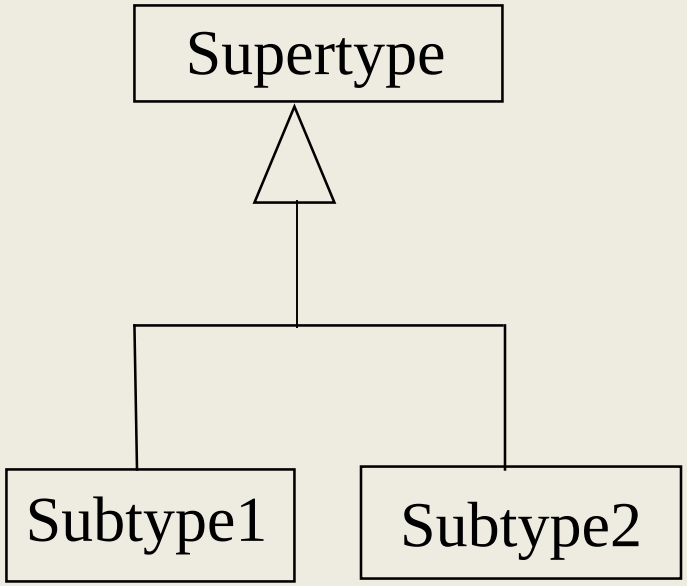
An example of Class



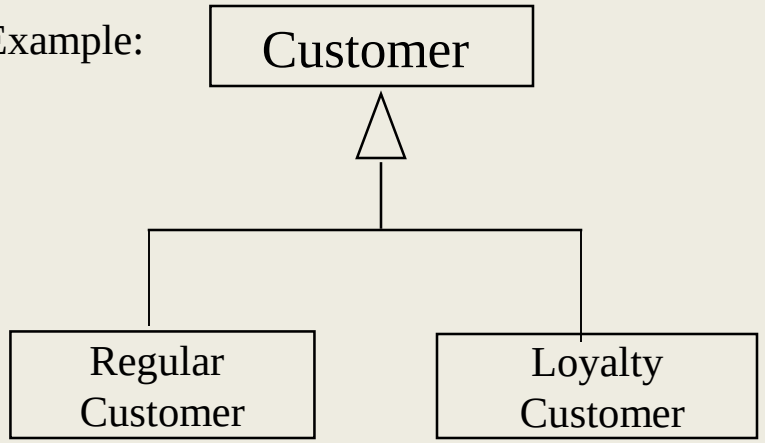
OO Relationships

- There are two kinds of Relationships
  - Generalization (parent-child relationship)
  - Association (student enrolls in course)

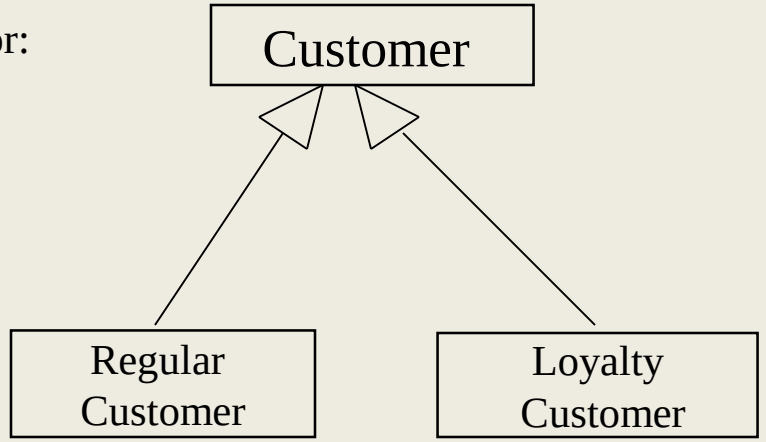
OO Relationships: **Generalization**



Example:



or:



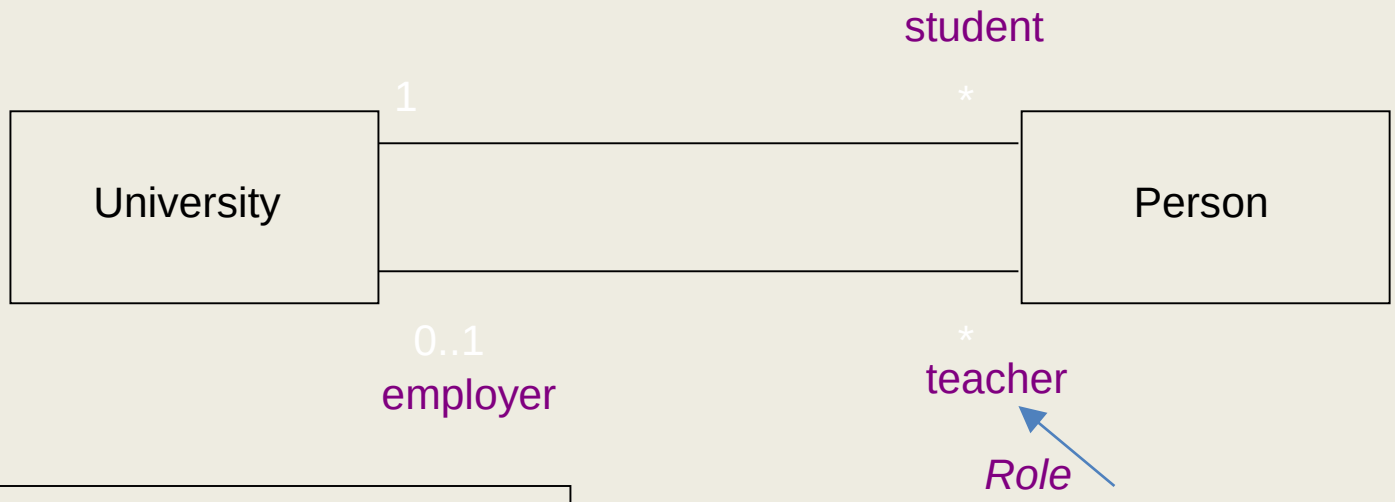
- Generalization expresses a parent/child relationship among related classes.

- Used for abstracting details in several layers

OO Relationships: **Association**

- Represent relationship between instances of classes
  - Student enrolls in a course
  - Courses have students
  - Courses have exams
  - Etc.
- Association has two ends
  - Role names (e.g. enrolls)
  - Multiplicity (e.g. One course can have many students)
  - Navigability (unidirectional, bidirectional)

Association: Multiplicity and Roles



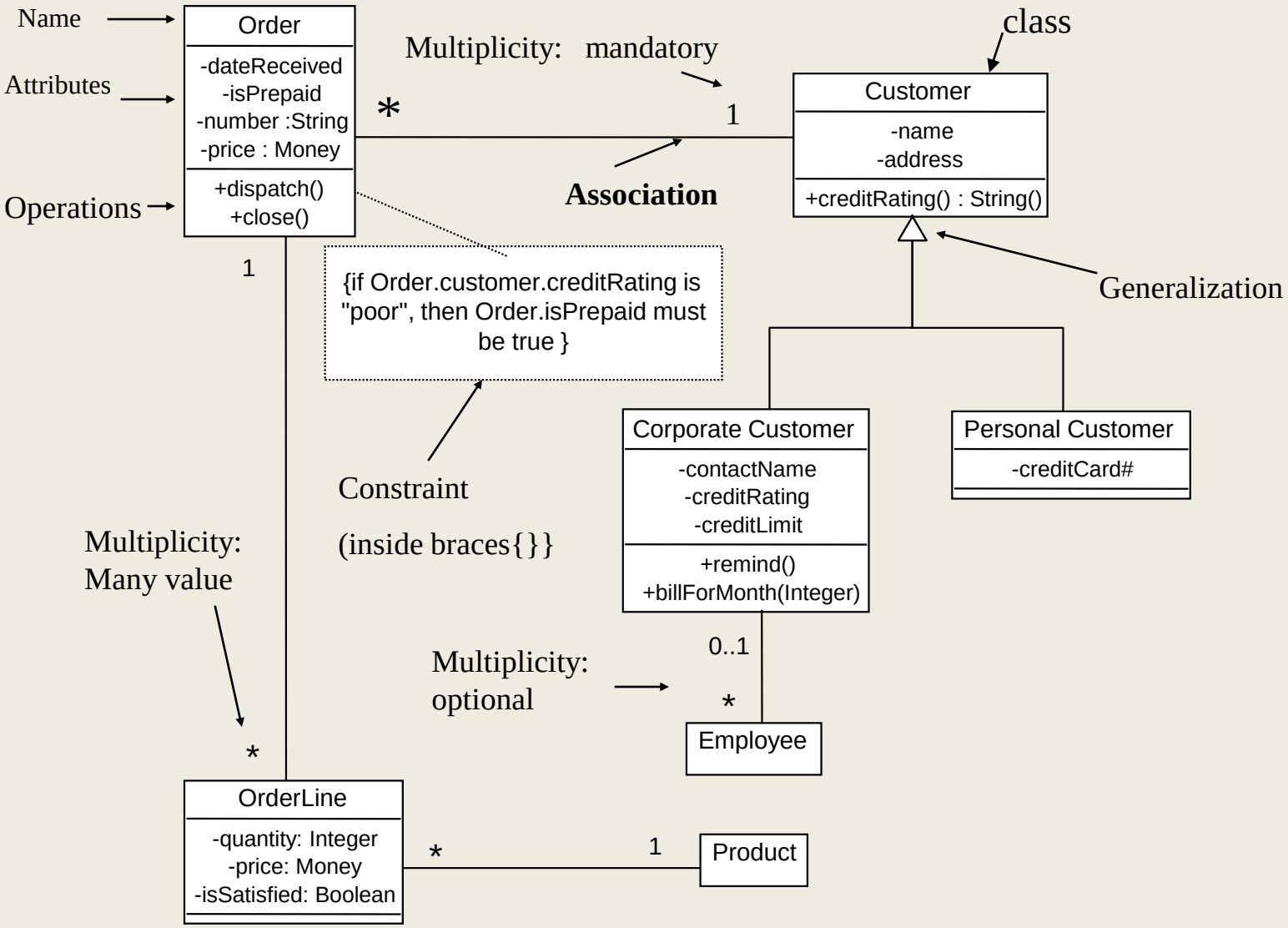
Multiplicity

Symbol	Meaning
1	One and only one
0..1	Zero or one
M..N	From M to N (natural language)
*	From zero to any positive integer
0..*	From zero to any positive integer
1..*	From one to any positive integer

Role

"A given university groups many people; some act as students, others as teachers. A given student belongs to a single university; a given teacher may or may not be working for the university at a particular time."

Class Diagram



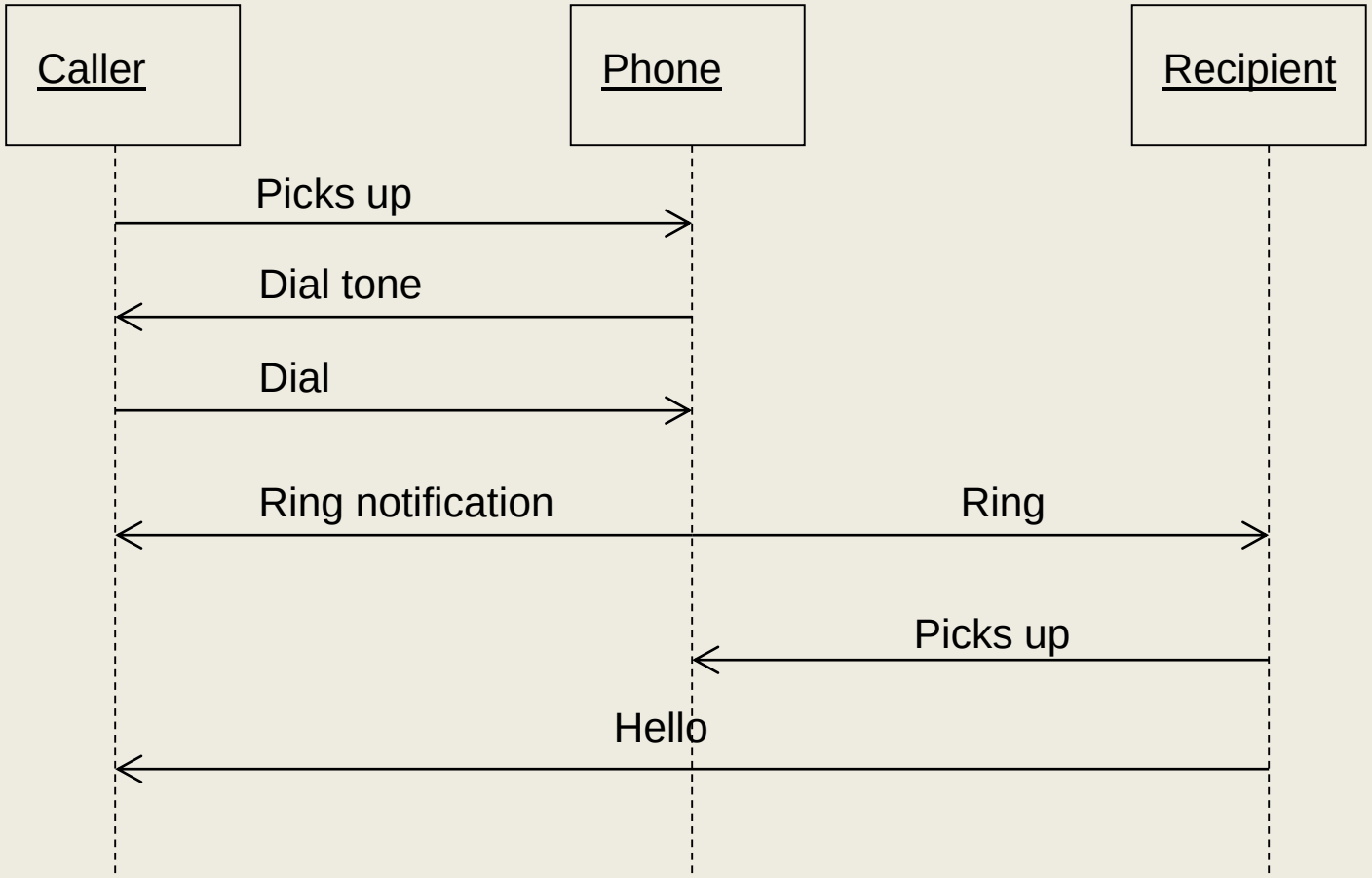
[from *UML Distilled Third Edition*]

Association: Model to Implementation


```
Class Student {
   Course enrolls[4];
}

Class Course {
    Student have[];
}
```

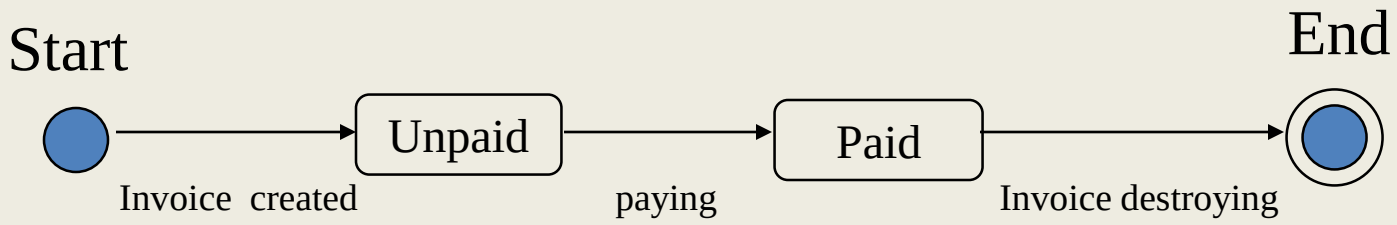
Sequence Diagram(make a phone call)



State Diagrams

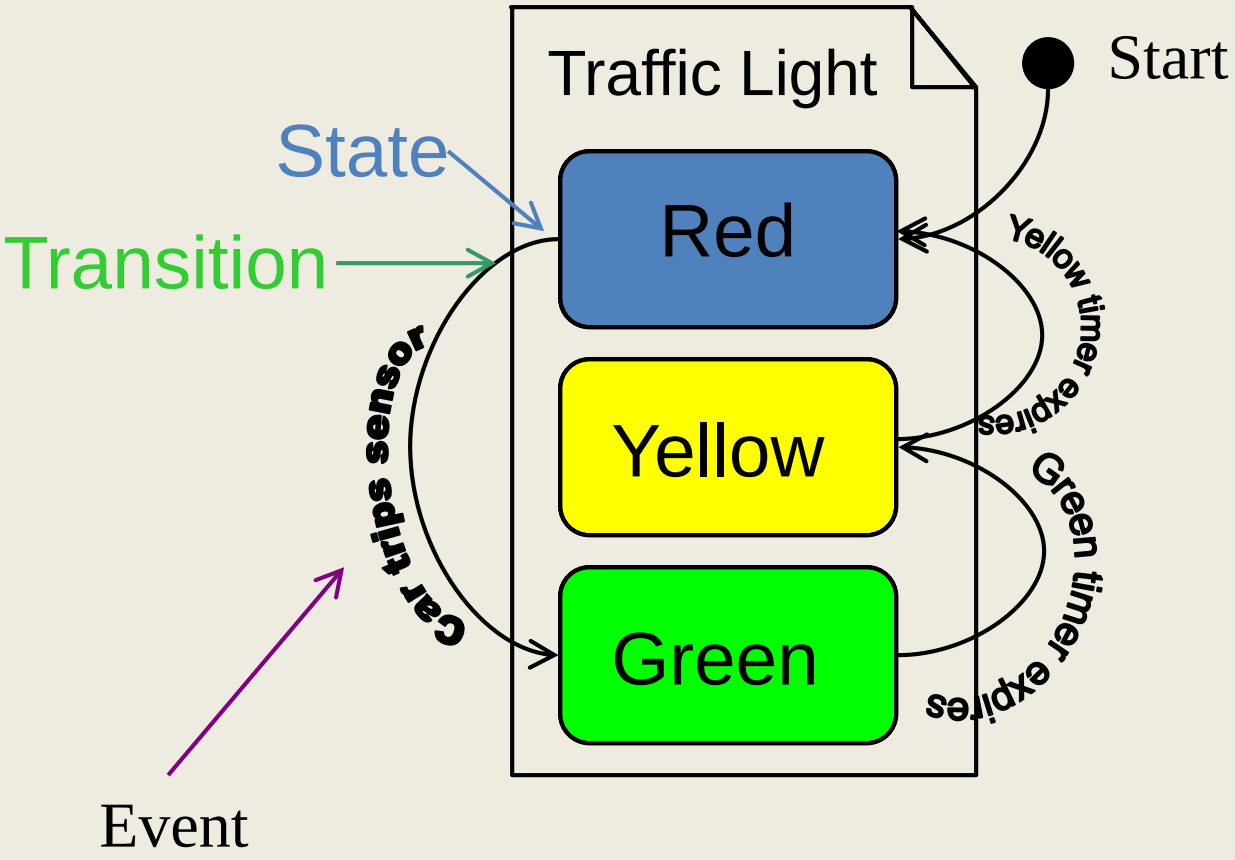
(Billing Example)

State Diagrams show the sequences of states an object goes through during its life cycle in response to stimuli, together with its responses and actions; an abstraction of all possible behaviors.



State Diagrams

(Traffic light example)



Activity Diagram

- Explained on the board in the class

UML Modeling Tools

- Rational Rose (www.rational.com) by IBM
- TogetherSoft Control Center, Borland (http://www.borland.com/together/index.html)
- **ArgoUML** (free software) (http://argouml.tigris.org/)
  OpenSource; written in java
- Others (http://www.objectsbydesign.com/tools/umltools_byCompany.html)

Reference

1. **UML Distilled:** A Brief Guide to the Standard Object Modeling Language
   Martin Fowler, Kendall Scott

2. IBM Rational
http://www-306.ibm.com/software/rational/uml/

3. Practical UML --- A Hands-On Introduction for Developers
   http://www.togethersoft.com/services/practical_guides/umlonlinecourse/

4. Software Engineering Principles and Practice. Second Edition;
   Hans van Vliet.

5. http://www-inst.eecs.berkeley.edu/~cs169/